

从 MongoDB 实践出发：如何构建 Agent 所需的数据基础平台

演讲人：周斌

MongoDB / 高级解决方案架构师

AiCon

全球人工智能开发与应用大会



企业 AI 之旅：我们在哪里？

购买

AI 驱动的代码编写是一个杀手级用例

购买

面向知识工作者的 Copilots 无处不在

构建

助力业务转型的 AI 智能体 (Agents)

我们的目标：帮助您大规模地将 **AI 智能体**投入到生产环境



AI正在重塑整个软件行业 and 软件架构

AI is Redefining Software Industry and Architecture

用户体验/界面层

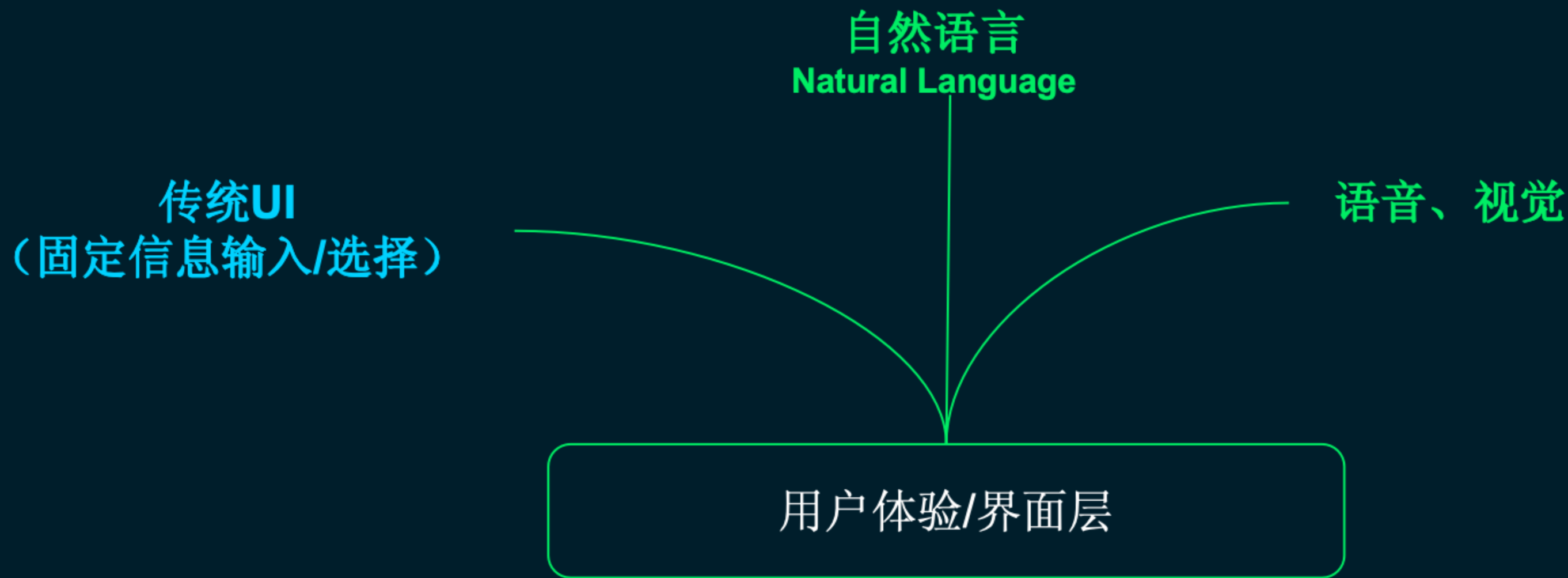
业务逻辑

数据基础



AI正在重塑整个软件行业 and 软件架构

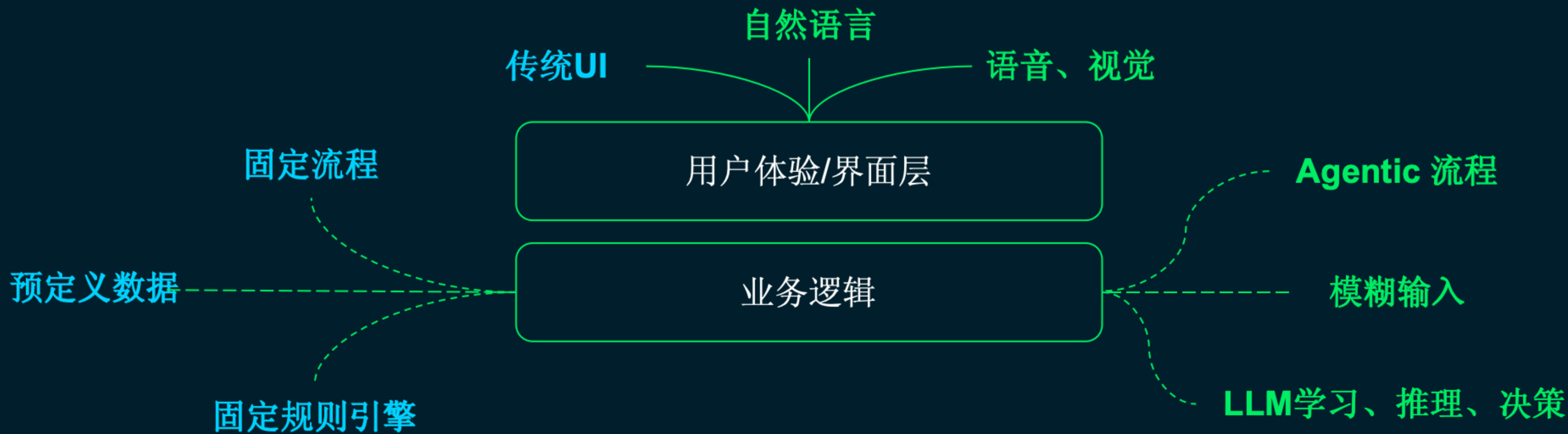
AI is Redefining Software Industry and Architecture





AI正在重塑整个软件行业 and 软件架构

AI is Redefining Software Industry and Architecture



AI正在重塑整个软件行业 and 软件架构

数据层如何改变？



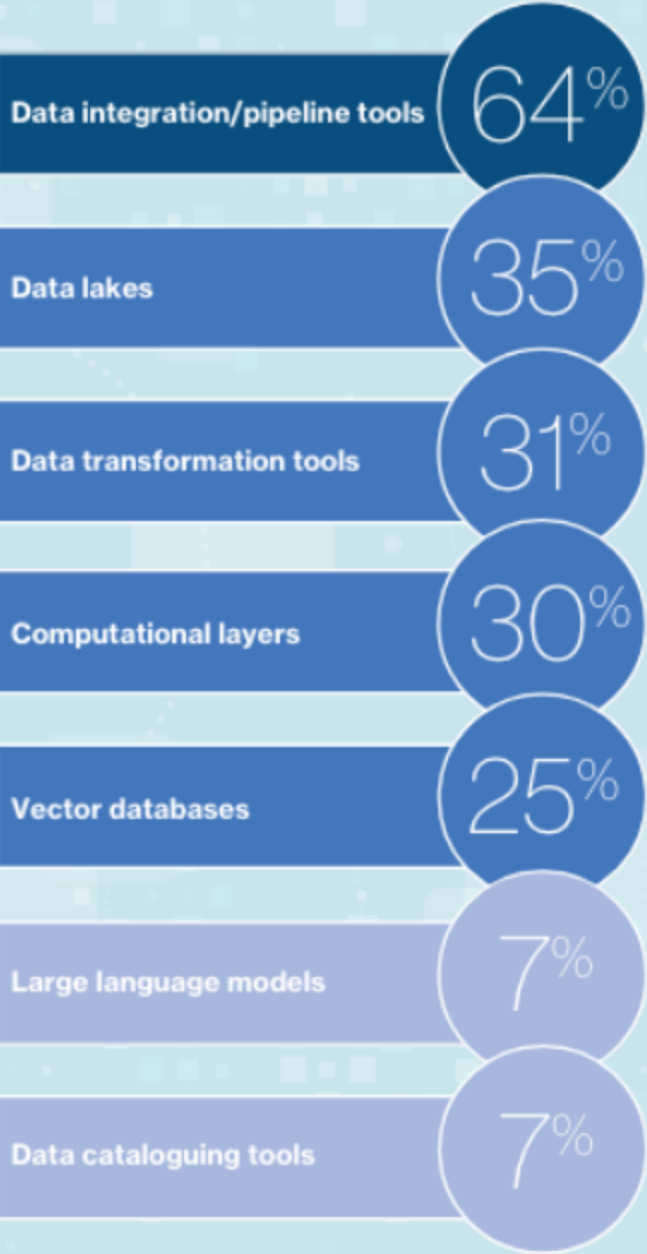
根据MIT的访谈，构建和部署AI，数据相关基础设施的投资比大模型的优先级更高，且比Vector数据库更重要

— AI readiness for C-suite leaders
MIT report



Figure 2. Top investment priorities when deploying AI

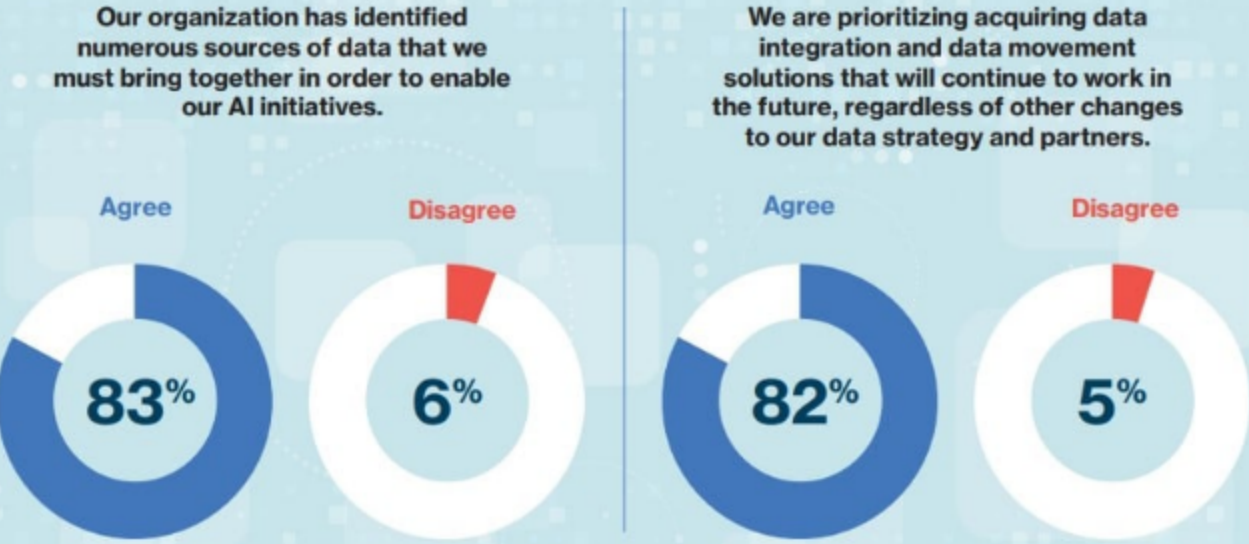
How would you prioritize the following areas of investment for an organization seeking to deploy AI? (Respondents ranking in top two.)



Source: MIT Technology Review Insights survey, 2024

Figure 1. Bringing data together is a top challenge

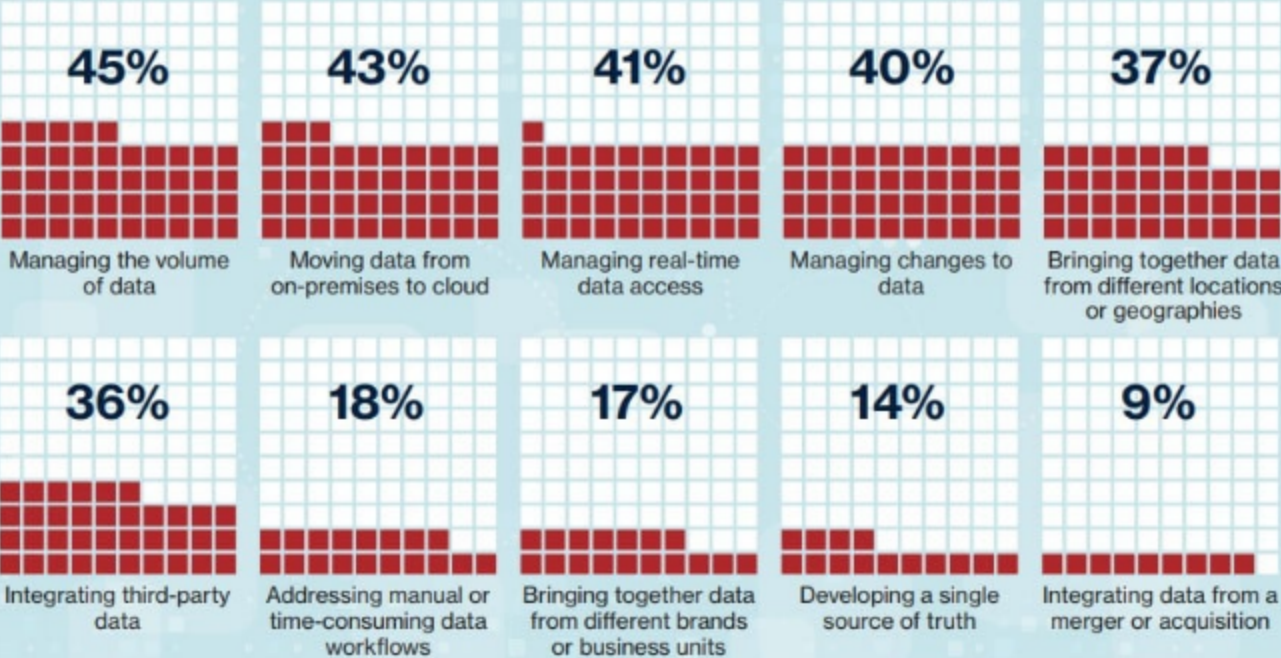
To what extent do you agree with the following statements?



Source: MIT Technology Review Insights survey, 2024

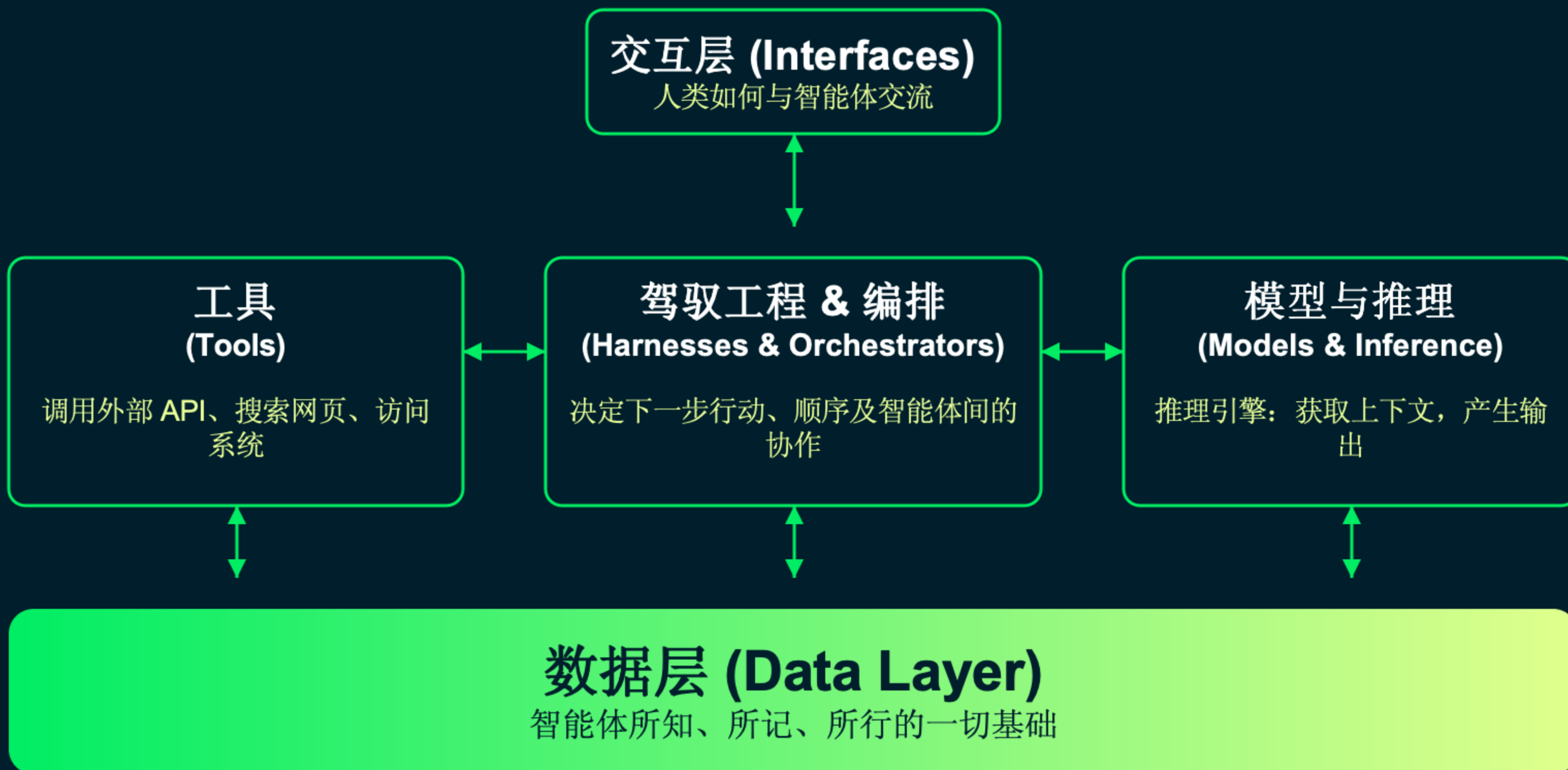
Figure 4. Numerous data integration challenges

What are the primary challenges your organization has encountered with data integration?



Source: MIT Technology Review Insights survey, 2024

智能体(Agents)技术栈





数据层决定了智能体 能否在**生产环境**中发挥作用

数据层 (Data Layer)

智能体所知、所记、所行的一切基础

企业级智能体成功关键： 数据够实时、够准确

实时且事务性



掌握当前发生的动态
专为智能体打造

离线且分析型



分析已发生的历史
专为报表打造



数据和数据处理的全新挑战

让我们从一个最简单**AI**服务场景入手



传统应用中的典型数据域，如电商

数据主要以“操作记录”的形式存在

数据域	关键信息
客户域 (Customer)	客户基本信息 认证信息、收货地址、客户360等
商品域 (Product)	类目、SPU/SKU、属性、库存、价格、评价
行为域 (Behavior)	点击流、搜索、曝光、埋点日志
订单域 (Order)	下单、支付、退款结算、购物车信息等
物流域 (Logistics)	运单号、轨迹、配送状态、快递公司管理
营销域 (Marketing)	优惠券、积分、秒杀、活动参与记录
风控域 (Risk)	黑名单、异常登录、反套现等



AI Agent 需要新一代的数据类型与处理范式

数据目的从“记录操作”变为“理解意图”，数据格式从“预定义”变为“动态涌现”





AI Agentic 服务将带来数据的根本变革

数据目的从“记录操作”变为“理解意图”，数据产生从“预定义格式”变为“动态涌现”



客户：“三月带5岁双胞胎去长野滑雪。第一次去，要准备什么，重点是把孩子带好，我左膝有旧伤，也别出事，最好经济实惠的”

AI Agent 需要新一代的数据类型与处理范式

AI agents demand a new class of data types and processing paradigms.

AI Agent 架构决策数据



客户：“三月带5岁双胞胎去长野滑雪。第一次去，要准备什么，重点是把孩子带好，我左膝有旧伤，也别出事，最好经济实惠的”

JSON 是事实上 Agent 与 LLM 交互的“母语”



无论提交给AI LLM的对话，还是LLM返回的对话，都是JSON

客户：“三月带5岁双胞胎去长野滑雪。第一次去，要准备什么，重点是把孩子带好，我左膝有旧伤，也别出事，最好经济实惠的”

```
import openai
from openai import OpenAI

client = OpenAI(
    api_key="YOUR_API_KEY",
    base_url="https://api.openai.com/v1" # 如果是Deepseek则换成其Base URL
)

chat_request = {
    "model": "gpt-4o",
    "messages": [
        {"role": "system", "content": "你是一位专业、贴心、诚实的AI购物推荐助手，名叫「购明智」 ..... "},
        {"role": "user", "content": "三月带5岁双胞胎去长野滑雪。第一次去，要准备什么 ..... "}
    ],
    "temperature": 0.7
}

response = client.chat.completions.create(**chat_request)
```

AI Agent 需要新一代的数据类型与处理范式

AI agents demand a new class of data types and processing paradigms.

AI Agent 架构决策数据



1. 长期画像 (Long-term Profile)

- 家庭成员： 5岁双胞胎
- 健康状况： 膝盖旧伤史
- 价格敏感度： 极度敏感（关注性价比）

AI Agent 需要新一代的数据类型与处理范式

AI agents demand a new class of data types and processing paradigms.

AI Agent 架构决策数据



事实记忆 (Session Context Fact & Intent)

- 目的地: 长野
- 任务: 家庭滑雪
- 技巧: 初学者
- 重点: 照顾孩子与安全

JSON 是事实上 Agent 与 LLM 交互的“母语”

无论提交给AI LLM的对话，还是LLM返回的对话，都是JSON



AI 基于对话提取的实事和意图，千差万别无法预先定义，只能有灵活的JSON可以完美支持

客户：家庭滑雪 | Client A: Family Skier

1. 长期画像 (Long-term Profile)

```
{  
  Persona: ["Protective Parent", "Novice Skier"],  
  Price_sensitivity: 0.95  
  Family: { "kids": "5-yo Twins" },  
  Health: "Knee Injury",  
}
```

2. 会话情境 (Session Context: Fact & Intent)

```
{  
  Dest: "Nagano, JP",  
  Task: "Family-Ski",  
  Focus: ["High-Visibility", "Anti-lost"],  
  Risk_mitigation: "Left-Knee-Support",  
  Deal_preference: "Family-Bundle"  
}
```

AI Agent 需要新一代的数据类型与处理范式

AI agents demand a new class of data types and processing paradigms.

AI Agent 架构决策数据



序号	产品名称	颜色	规格/款式	核心卖点与功能	适用人群/场景	
1	三合一冲锋衣	深蓝色	成人款（可拆卸）	耐脏实用、经济实惠，三合一设计灵活应对气温变化	成人 / 追求性价比及商务质感	
2	碳纤维加压护膝	黑色	专业防护款	提供强力的侧向支撑与加压防护，有效保护关节	成人 / 有膝盖旧伤、需强化防护者	
3	专业偏光滑雪镜	多色可选	近视友好款 (OTG)	防眩光、滤强光，内部空间大，可兼容佩戴近视眼镜	成人 / 近视滑雪爱好者	
4	护耳羊毛帽	深色系	极寒加厚款	选用高保暖羊毛，延伸护耳设计，强效抵御严寒	成人 / 极寒天气、头部防风保暖	

JSON 是事实上 Agent 与 LLM 交互的“母语”



无论提交给AI LLM的对话，还是LLM返回的对话，都是JSON

在AI 作出推荐前，要做语义搜索，这就要求对产品信息做向量化，为了提高查询的准确性，通常知识数据也是由JSON和向量vector混合组成的，**向量用来理解意图，JSON 用来定义属性**

```
[
{
  "product_id": "SKU-KIDS-7704", "name": "Fluorescent Orange Safety Jumpsuit",
  "category": "Kids / Apparel", "tags": ["High-Visibility", "Anti-Lost", "Jumpsuit"],
  "price": 599.00, "specs": { "color": "Fluorescent Orange", "waterproof": "15000mm" },
  "text": "荧光橙色儿童连体滑雪服，极致高可见度，防止在雪场丢失，全方位防水保暖。",
  "vector": [-0.56, 0.42, 0.89, -0.11, 0.67, ..... ]
},
{
  "product_id": "SKU-MED-3302", "name": "Hinged Knee Brace Pro",
  "category": "Medical / Sports-Protection", "tags": ["Knee-Support", "Injury-Recovery", "Heavy-Duty"],
  "price": 320.00, "medical_specs": { "support_level": "Level-3 (Maximum)", "material": "Medical-grade Neoprene",
  "applicable_joints": ["Left-Knee", "Right-Knee"] }, "available_sizes": ["S", "M", "L", "XL"],
  "text": "医用级双侧弹簧支撑运动护膝，专为韧带损伤设计，提供强效减震与骨骼对齐保护，滑雪防再次受伤必备。",
  "vector": [0.12, -0.78, 0.45, 0.91, -0.23, ..... ]
}
]
```

JSON 是事实上 Agent 与 LLM 交互的“母语”

无论提交给AI LLM的对话，还是LLM返回的对话，都是JSON



AI Agent 将客户的要求，与更新后的Profile、Fact 以知识库选出的结果，用JSON格式作为Context，再提交给LLM Model

```
# 1. 模拟从知识库（向量/关系数据库）中检索出来的符合条件的最新产品池
retrieved_products_from_db = [ { "product_id": "SKU-MED-3302", ..... }, { "product_id": "BUNDLE-NAGANO-002", }]

# 2. 构造发送给大模型的 context_aware_request 呼叫
response = openai.Chat.completion.create(
    model="gpt-4o",
    temperature=0.2, # 低随机性，保证 JSON 输出稳定
    response_format={"type": "json_object"}, # 强制大模型必须输出标准的 JSON 格式
    messages=[
        {
            "role": "system", "content": f"""你是一位专业的电商专家。你的任务是根据用户的最新反馈、用户画像、以及知识库返回的产品列表，为用户生成一个定制化的Offer。
            # 【当前用户画像 (User Profile)】
            - {User Profile.....}
            - {Intent..... }
            # 【知识库最新召回的相关产品 (Retrieved Products)】
            这是系统刚刚为你从商品库中检索出来的合格商品，你只能从这里面选择：
            {retrieved_products_from_db}
            """
        },
        # 用户的最新动态反馈（打破了上一次的方案）
        { "role": "user", "content": "三月带5岁双胞胎去长野滑雪。第一次去，要准备什么 ....." },
        # 系统指令：催促大模型进行最终合并和输出
        { "role": "system", "content": "请立刻结合上述所有上下文和知识库产品，更新并输出最终的 JSON Offer。" }
    ]
)
```

AI Agent基于当前对话、实事、偏好，以及产品知识库，动态定制产品组合



AI Agent 架构决策数据

最新的对话
Context Window

用户偏好
Core Memory/ User Profile

实事记忆
Semantic / Factual
Memory

情景记忆
Episodic Memory /
Summary

产品知识库

AI Provides Package Offer : “长野首滑”礼包



客户： 不错，帮我加进购物车

JSON 是事实上 Agent 与 LLM 交互的“母语”



无论提交给AI LLM的对话，还是LLM返回的对话，都是JSON

Agent 与传统 RAG 的分水岭在于‘**业务决策**’。决策要求 **AI 具备对业务数据进行动态建模，实时生成订单、动态修改套餐、完善用户画像**。传统的预定义模式（Fixed Schema）会成为 AI 灵活性的枷锁；而 **JSON** 因为其灵活性，已经成为了 AI 自主决策的数据事实标准

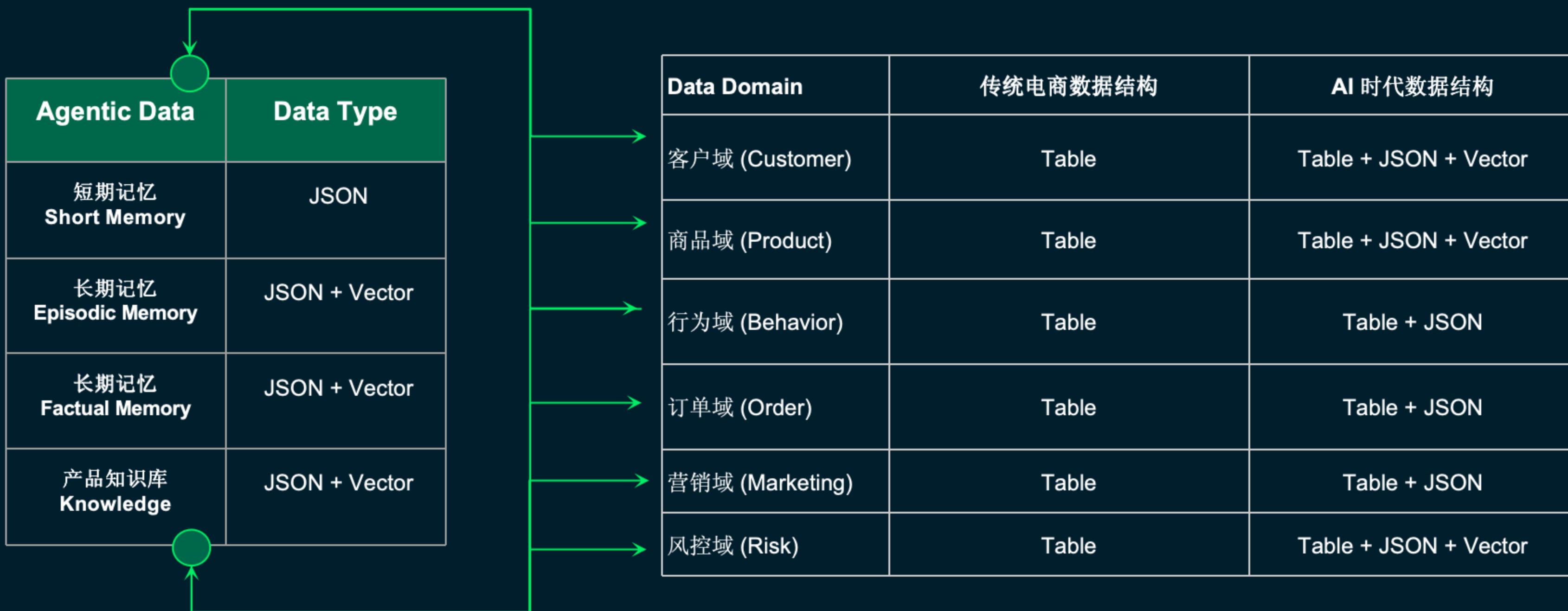
```
{
  "offer_id": "OFFER-NAGANO-2026-001",
  "intent": "Family_Safe_Skiing",
  "context": { "trip_des": "Nagano, Japan", "persona": "3-Person Family (Adults with injury history + Young kids)" },
  "decision_logic": { "safety_priority": "High-Visibility & Joint-Protection", "bundle_type": "Nagano First-Ski Package"},
  "items": [
    {
      "group": "Adult / Pro-Protection",
      "skus": [{ name: "SKU-NAVY-3IN1" ... }, {name: "SKU-PRO-KNEE-GUARD",.....}, ..... ],
      "agent_note": "针对成人旧伤强化侧向支撑，深蓝色系兼顾耐脏与质感。"
    },
    {
      "group": "Kids / High-Visibility",
      "skus": [{name:"SKU-ORANGE-JUMP",.....},{name: "SKU-KIDS-GOGGLE",.....}, .... ],
      "agent_note": "通过荧光橙与反光设计确保视野盲区可见，极致防丢。"
    }
  ],
  "financial_decision": { "base_price": 4500, "strategy": "Family_Bundle_20%_OFF", "final_total": 3600, "currency": "CNY" },
  "metadata": { "status": "Generated_By_Agent", "flexibility_score": 0.95}
}
```



AI Agent 需要新一代的数据类型与处理范式

数据目的从“记录操作”变为“理解意图”，数据格式从“预定义”变为“动态涌现”

JSON 和 Vector 是新一代AI应用的数据关键变化



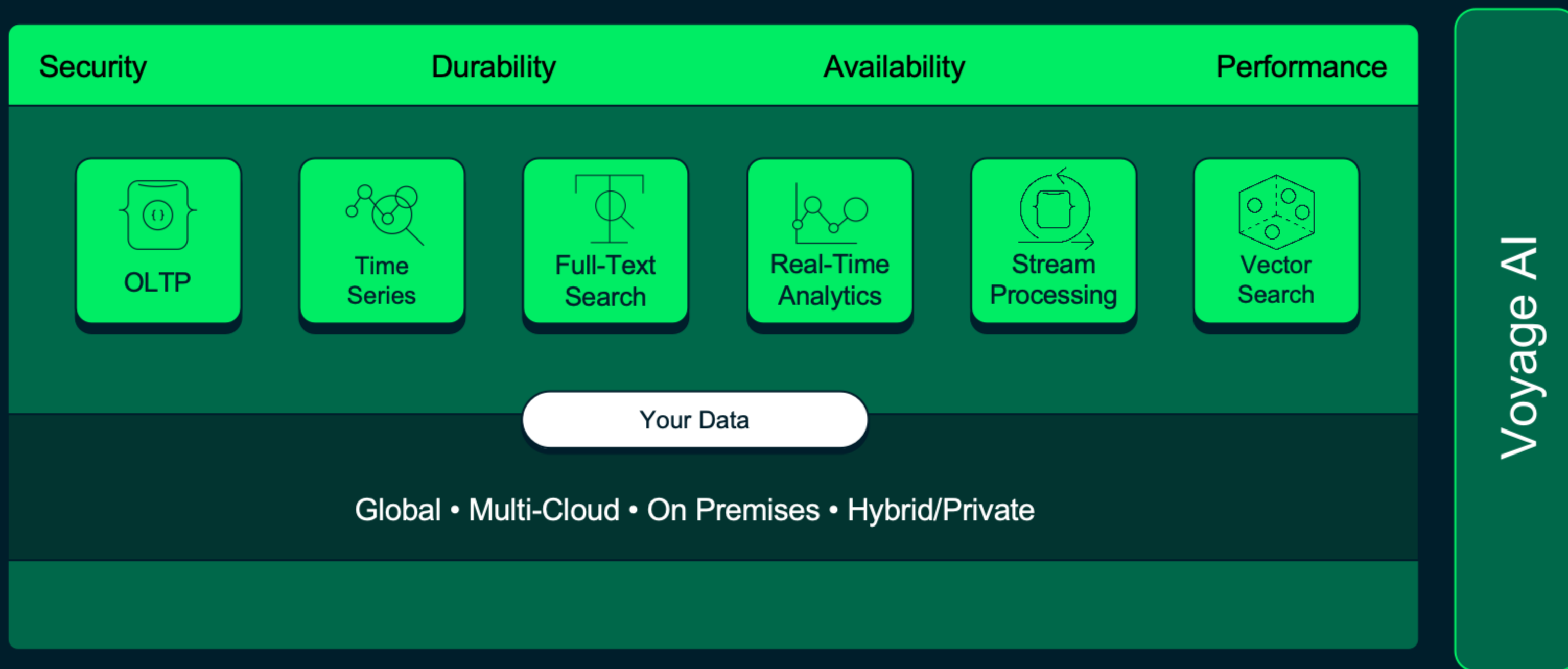
AI的数据层必须支持多模态数据的 从**CRUD**到记忆体的全栈闭环



MongoDB: AI Ready的数据平台



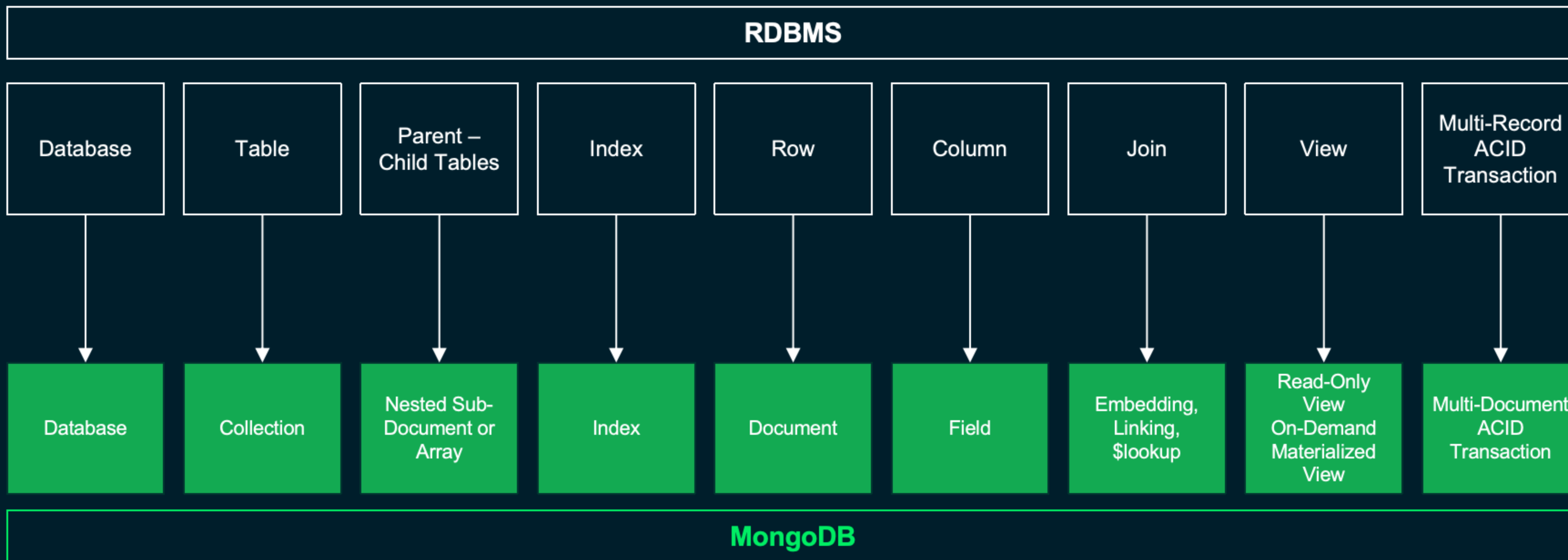
除了OLTP必须的基于ACID的增删改查，还有原生的地理位置、时序数据、全文搜索、向量搜索、Embedding & Reranking 等各类AI需要的数据操作





MongoDB兼具分布式数据库特性与传统数据库的功能

传统数据库，如PostgreSQL能到做的数据CRUD操作， MongoDB能做！





MongoDB与能够支持分布式跨表事务，保证数据一致性

MongoDB和RDBMS一样可以支持Transaction，而且可以支持分布式Transaction



```
START TRANSACTION;  
INSERT INTO orders (...) VALUES (...);  
  
UPDATE stock  
  SET quantity = ...  
  WHERE ...;  
  
COMMIT;
```



```
session.startTransaction();  
db.orders.insert ({...});  
db.stock.update (  
    {... },  
    {$set: { quantity: ...} }  
);  
session.commitTransaction();
```



JSON是AI原生数据格式， 而MongoDB是为JSON而生的原生数据库

与其他数据库将JSON作为辅助数据类型不同，JSON是MongoDB产品的核心，其原生存储格式就是BSON (二进制 JSON)，因此最擅长处理AI数据。



MongoDB 是一种面向文档的 **OLTP** 数据库，并且原生支持**分布式数据**架构。

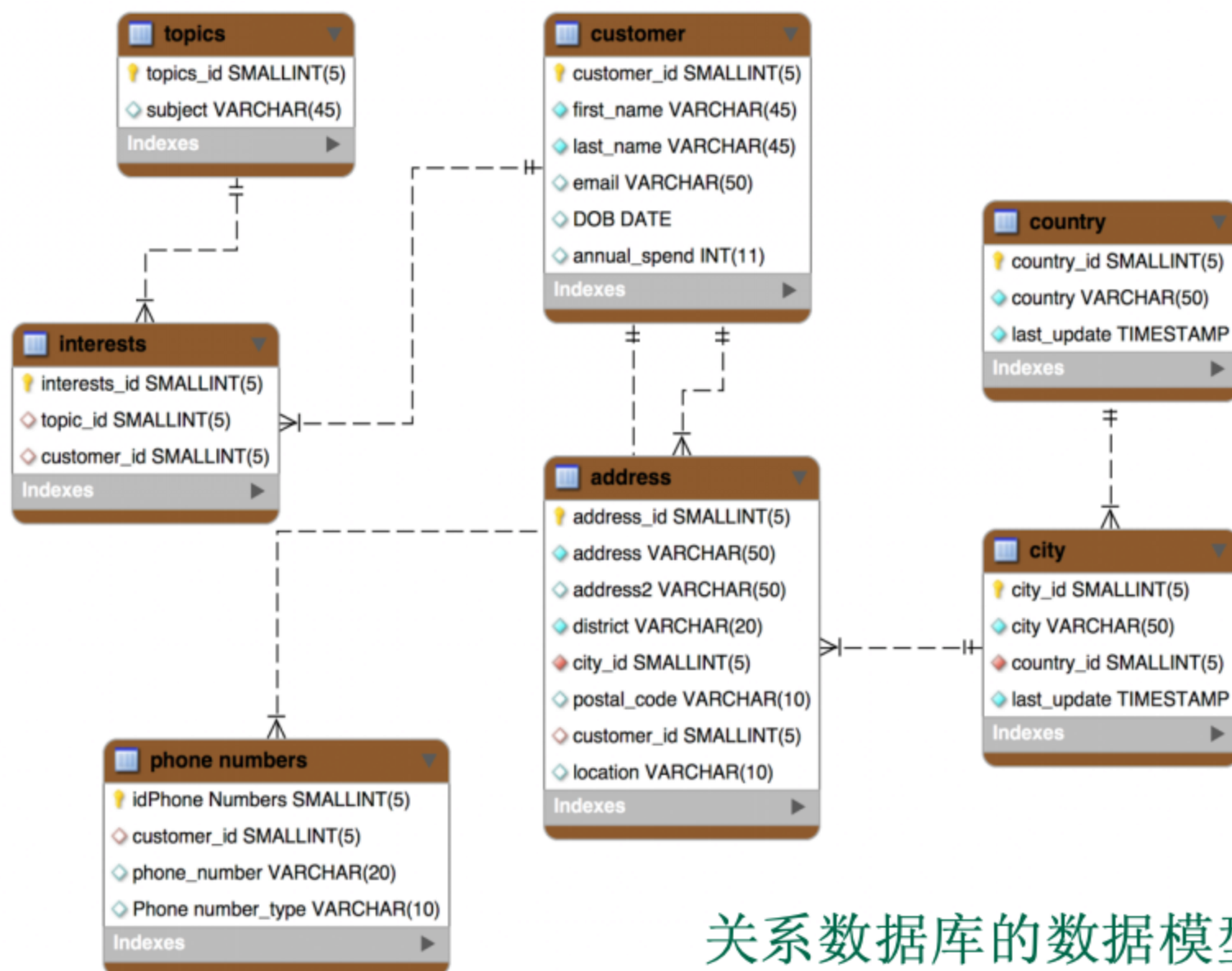
- ▮ 文档是一种灵活的类 JSON 结构（BSON）
 - 集合没有强制的模式
 - 非常适合数据聚合、多渠道通信、数据集成、多步工作流等场景
- ▮ **OLTP数据库**针对数据的随机访问进行了优化
 - 提供强一致性保证
 - 点查询和范围查询都非常快速
- ▮ **分布式数据架构**是一种可在多台服务器之间复制和/或分发数据的技术
 - 高可用性：没有单点故障
 - 横向可扩展



INTUITIVE Document Database

Related data contained in a single, rich document

```
{
  "_id" : ObjectId("5ad88534e3632e1a35a58d00"),
  "name" : {
    "first" : "John",
    "last" : "Doe" },
  "address" : [
    { "location" : "work",
      "address" : {
        "street" : "16 Hatfields",
        "city" : "London",
        "postal_code" : "SE1 8DJ"},
      "geo" : { "type" : "Point", "coord" : [
        -0.109081, 51.5065752]}}],
    + {...}
  ],
  "dob" : ISODate("1977-04-01T05:00:00Z"),
  "retirement_fund" : NumberDecimal("1292815.75")
}
```



关系数据库的数据模型
Related data split across multiple
records and tables

Document Data Model

Related data contained in a single, rich document

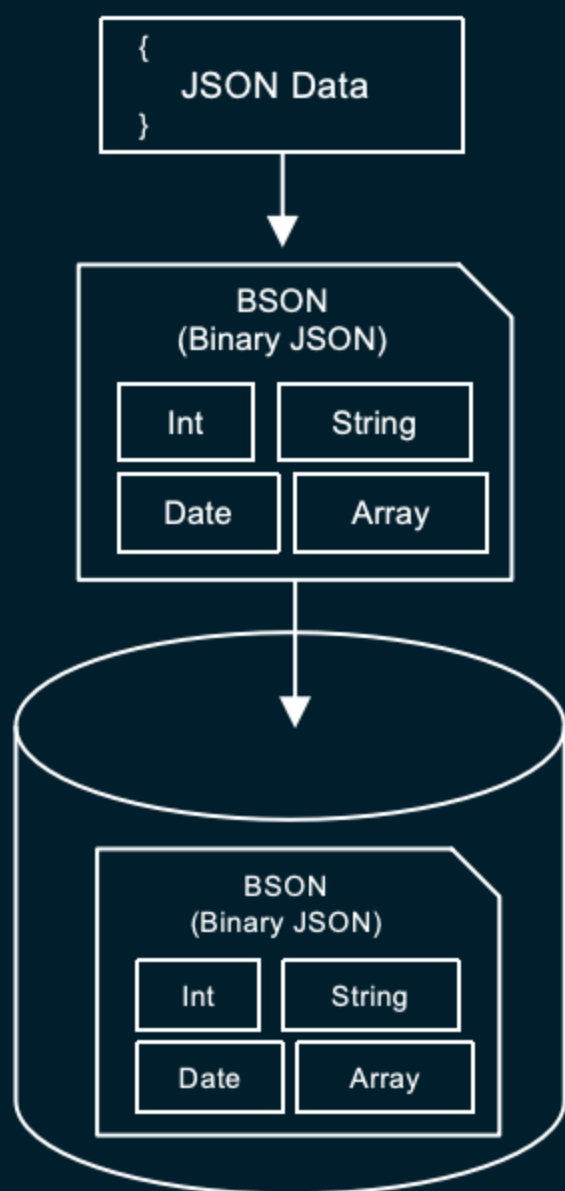
```
{
  _id : ObjectId(5ad88534e3632e1a35a58d00),
  name : {
    first : John,
    last : Doe },
  address : [
    { location : work,
      address : {
        street : 16 Hatfields,
        city : London,
        postal_code : SE1 8DJ},
      geo : { type : Point, coord : [
        -0.109081, 51.5065752]}}],
  + {...}
],
  dob : ISODate(1977-04-01T05:00:00Z),
  retirement_fund : NumberDecimal(1292815.75)
}
```

除了数据模型的区别，我们再看底层JSON存储的区别



MongoDB 存储方式

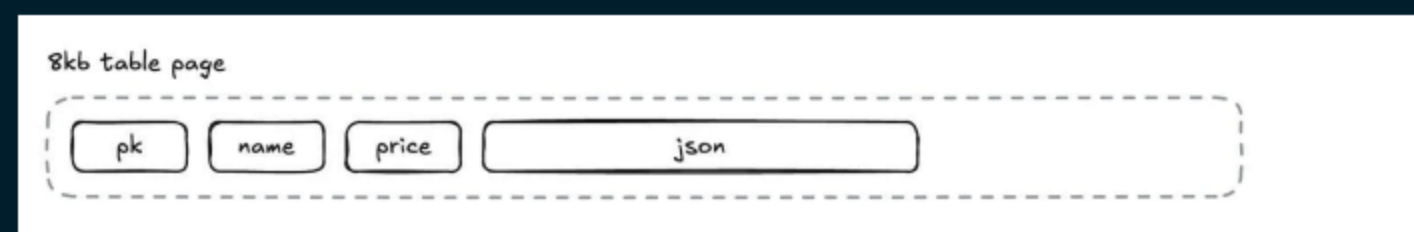
BSON, Stored as as Single Document



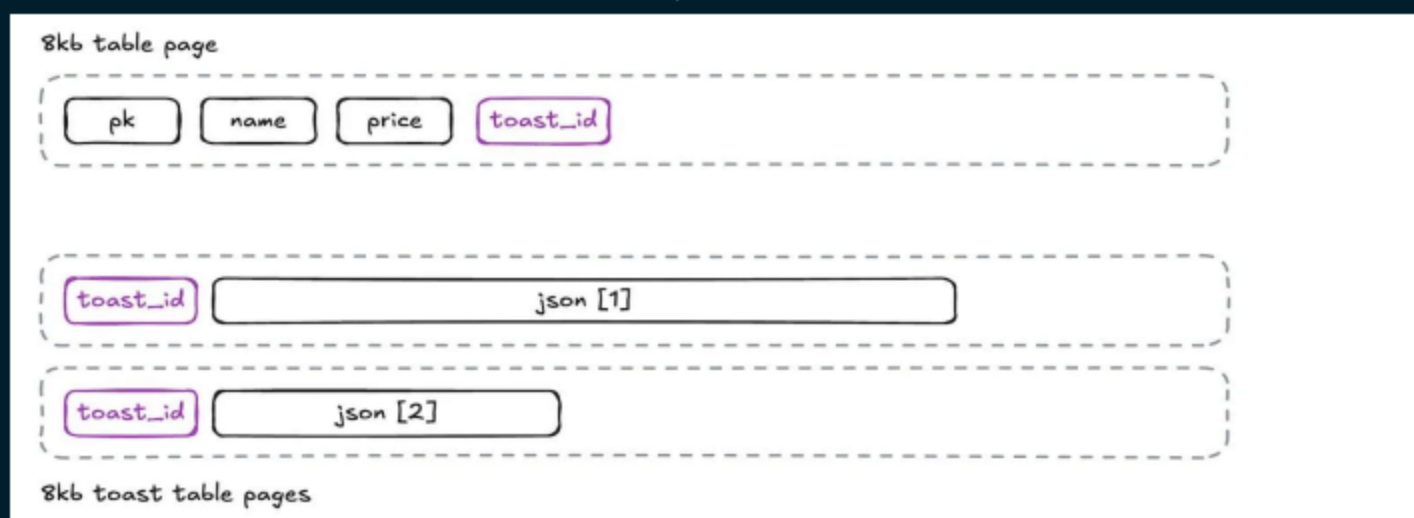
PostgreSQL JSON存储方式

TOAST, 超过2K, 切块存到外部表

< 2KB



> 2KB





当我们更新JSON，哪怕只Update了一个value时

阶段	PostgreSQL (TOAST + GIN Index)
1. 读取	读整行 + TOAST表，重构整份 jsonb
2. 修改	UPDATE 生成新一份数据（tuple），整行拷贝
3. 定位	新 tuple 新 TID，索引要随 TID 变
4. 写入	新 tuple + WAL，且大 jsonb 需TOAST 拆分 / 重写 chunk，再由 VACUUM 回收 dead tuple
5. 索引	jsonb 依赖 GIN 倒排：键/值 → TID，一次 UPDATE 必须 触发该 jsonb 内所有键/值索引更新

后果 很大写放大：一点改动，全量磁盘写入 + 大量索引修改

Although PostgreSQL scales well for our read-heavy workloads, **we still encounter challenges during periods of high write traffic**. This is largely due to PostgreSQL's multiversion concurrency control (MVCC) implementation, which makes it less efficient for write-heavy workloads. For example, when a query **updates a tuple or even a single field**, the entire row is copied to create a new version. Under heavy write loads, **this results in significant write amplification. It also increases read amplification**, since queries must scan through multiple tuple versions (dead tuples) to retrieve the latest one.

– From OpenAI Blog

Source: <https://openai.com/index/scaling-postgresql/>

在处理JSON的基础CRUD上，MongoDB是应对AI需求最佳平台



	MongoDB	传统数据库，如 PostgreSQL	为智能体业务带来的优势
数据的类型	超过15种，格式验证十分简单	只有4种，格式验证复杂且很影响性能	更少出错
查询和索引	按“对象.对象”操作	SQL + JSON 操作符混合，很难阅读	简单易用
	一个索引处理查询与排序	JSON用的Gin索引无法排序与B-tree索引难以融合	性能更高
存储的方式	BSON 整存整取	TOAST 超过2K，切块存到外部表	读写更快
扩展的能力	分布式架构，扩缩都不停服	单机架构，开发时就要思考分库策略，且难以缩容	自由扩缩

MongoDB处理AI的JSON数据，容错更好，功能更多、性能更佳，扩容更易且这些优势，都不是其他数据库可以通过AI Vibe Coding来提升的



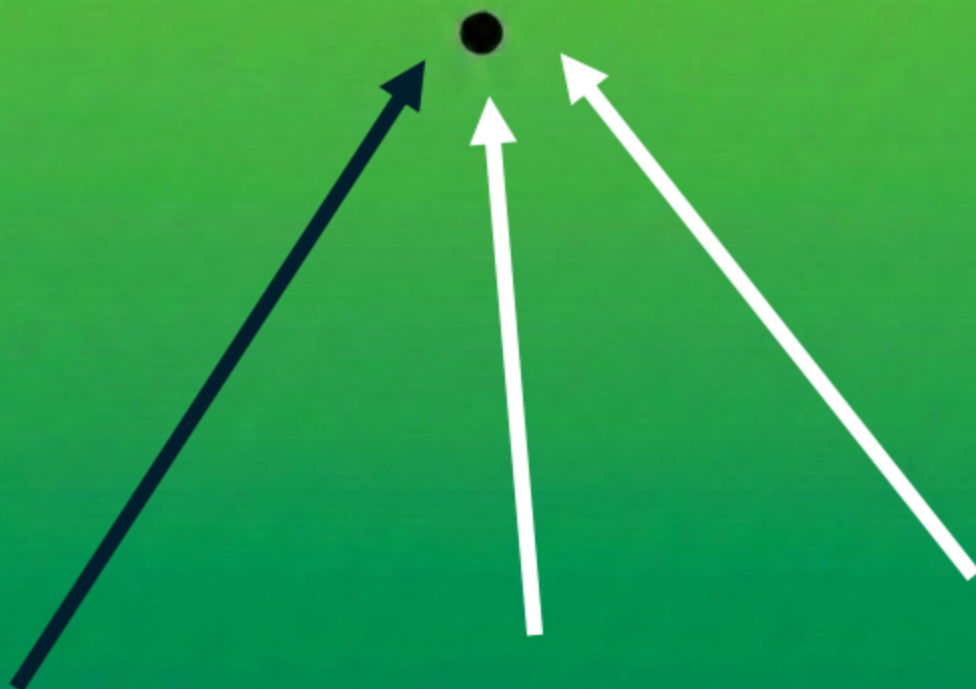
settings 非 LLM 问题

psychology 而是数据 检索 问题

MongoDB 核心优势



精准度



随处运行

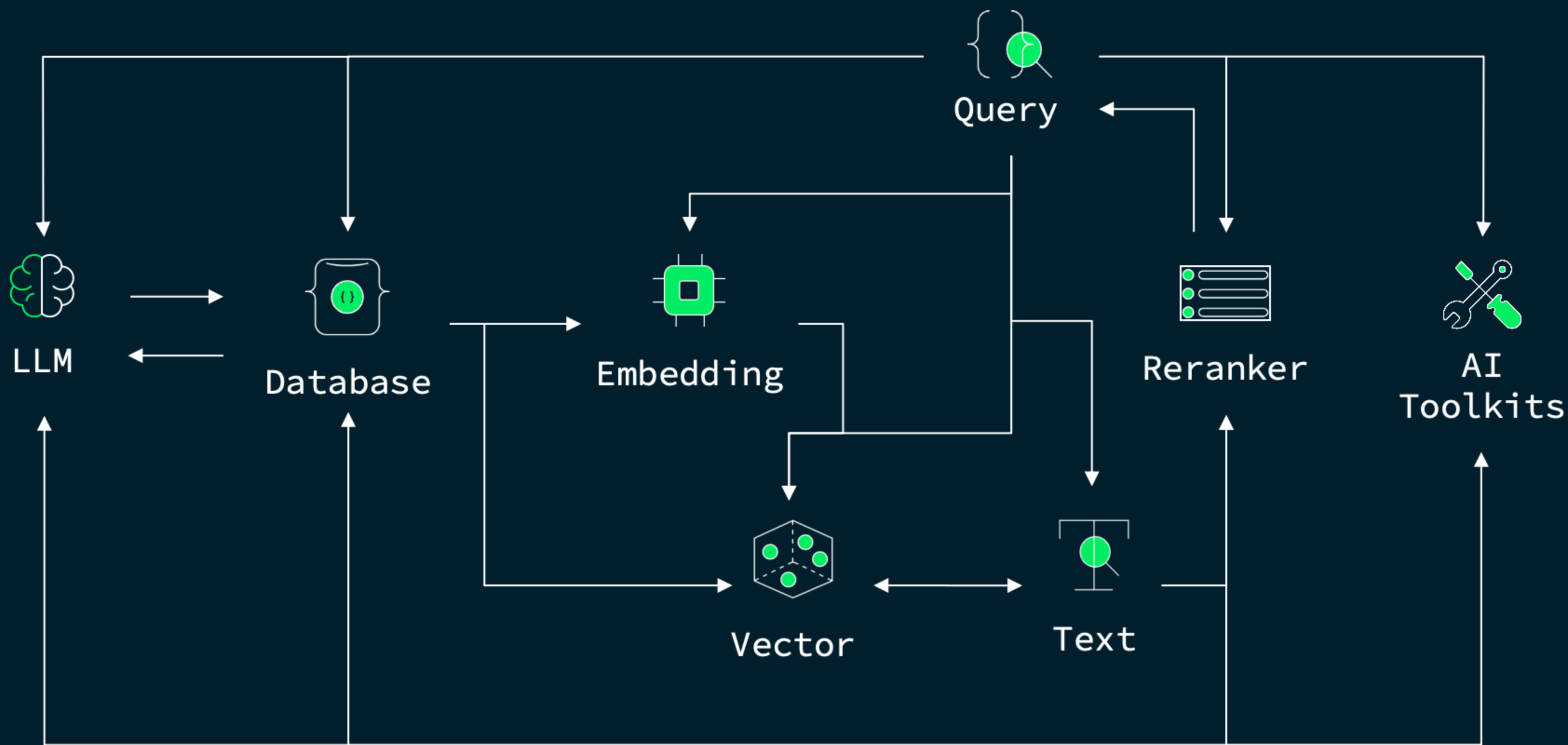


高性能、规模化、
高可用性





一次AI语义查询的路径



从CRUD到记忆体的数据全栈闭环

看起来容易，但实际上挑战巨大



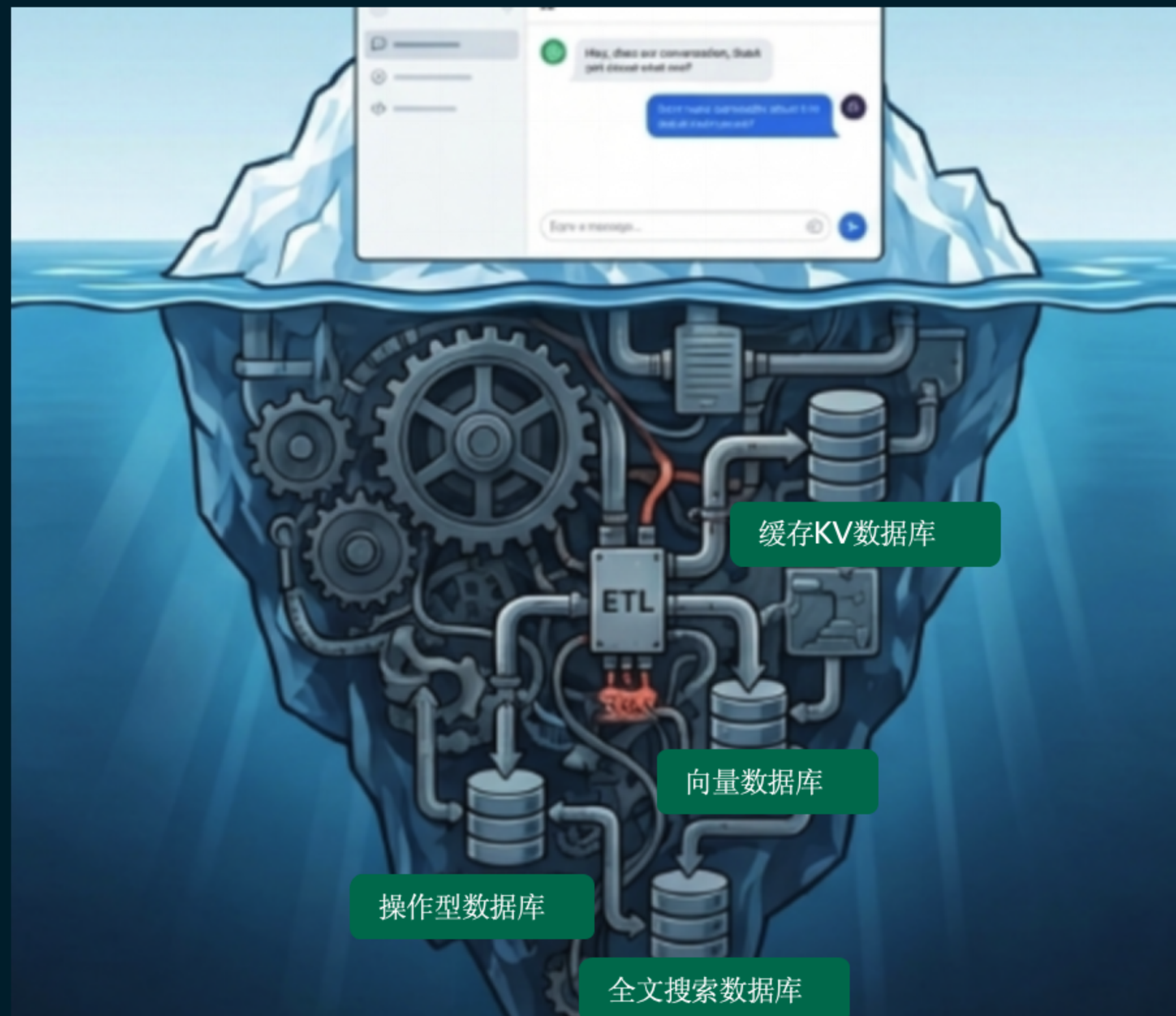
水面之上（表象）

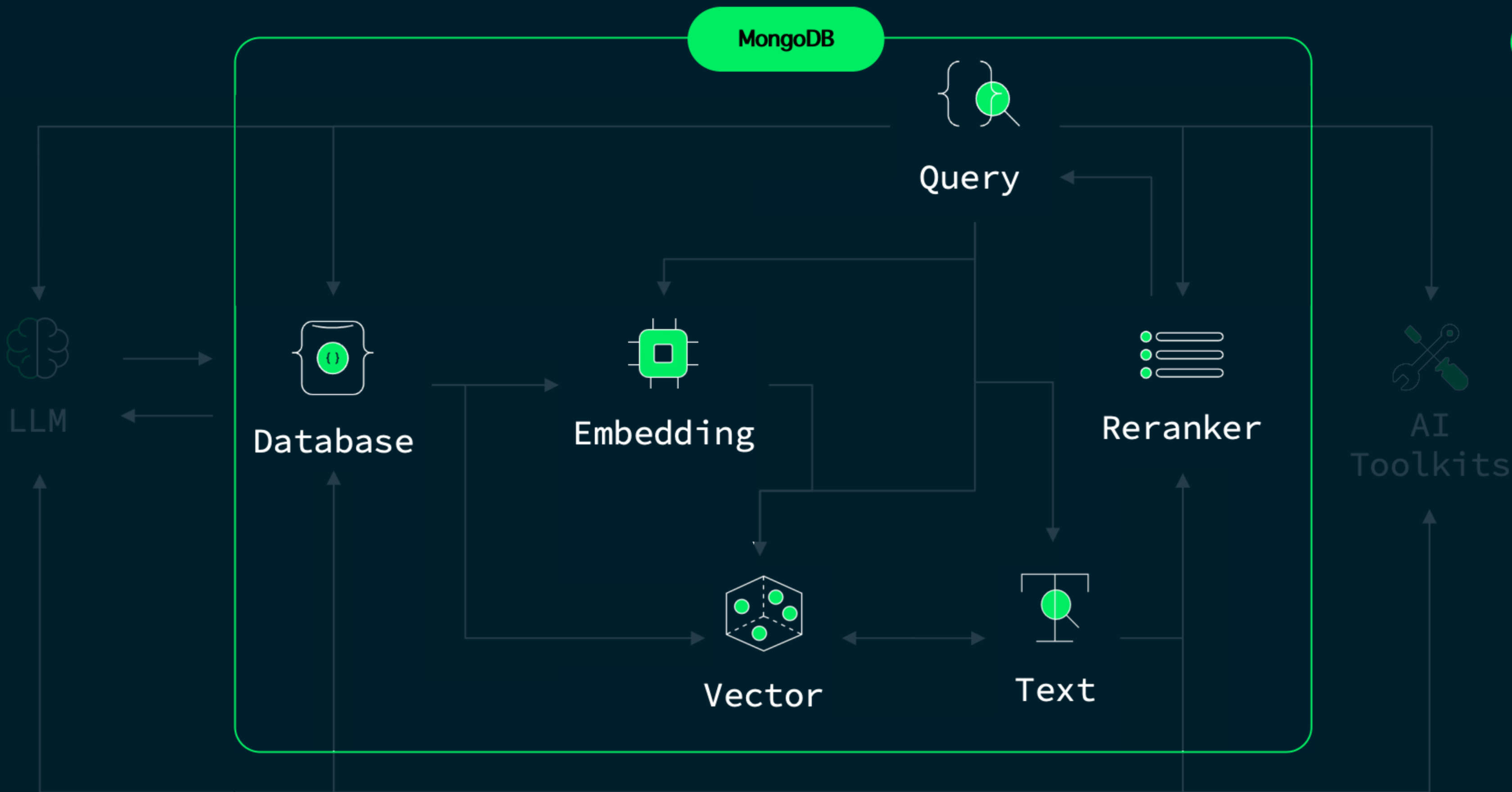
LLM和提示词工程，看似简单的AI对话调用。

水面之下（实际）

- 分离的系统：操作型数据库 + 向量数据库 + 全文搜索数据库 + 缓存数据库，通过ETL进行同步
- 数据一致性：文档状态更新，但向量/全文/缓存都要代码实现同步更新
- 扩展问题：传统数据库分片困难，难以横向扩容

核心结论：传统分离的数据库架构很难满足现代AI应用需求





MongoDB 原生的将这些能力模块整合在一个平台里，并通过**统一API**来操作

MongoDB 是智能体应用的理想统一数据平台

MongoDB 的最大特点之一，就是在同一个库中，通过统一的API来支持各种多模数据存储与操作

MongoDB 统一数据平台能力

storage

CRUD & ACID

search

全文搜索

shutter_speed

向量搜索

time line

时序数据

place

地理位置

waves

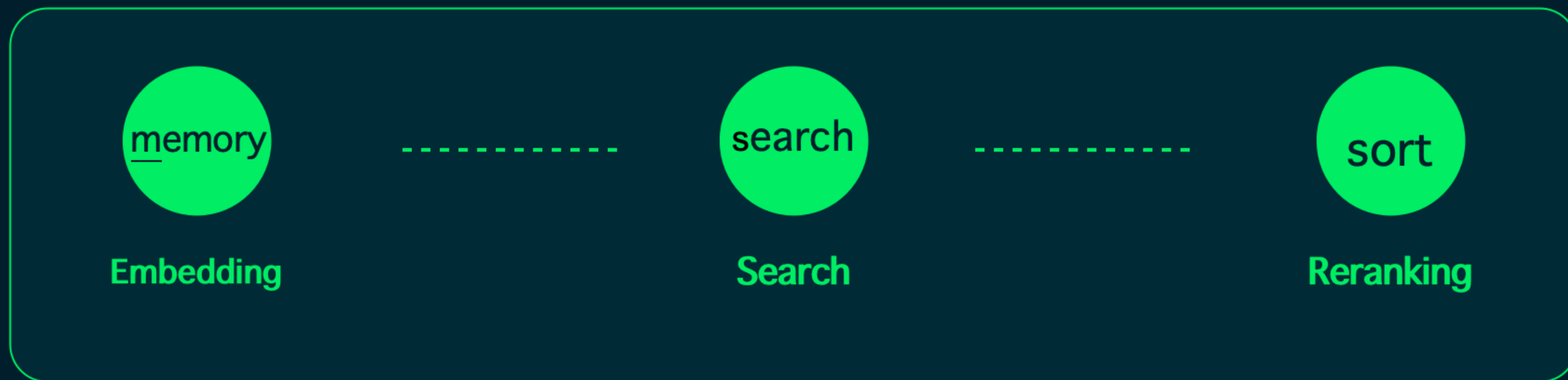
流处理

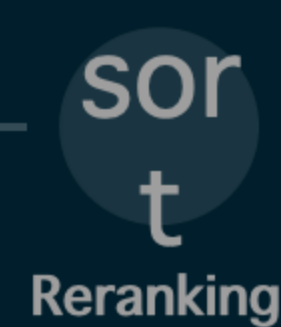
psychology

Embedding &
Reranking

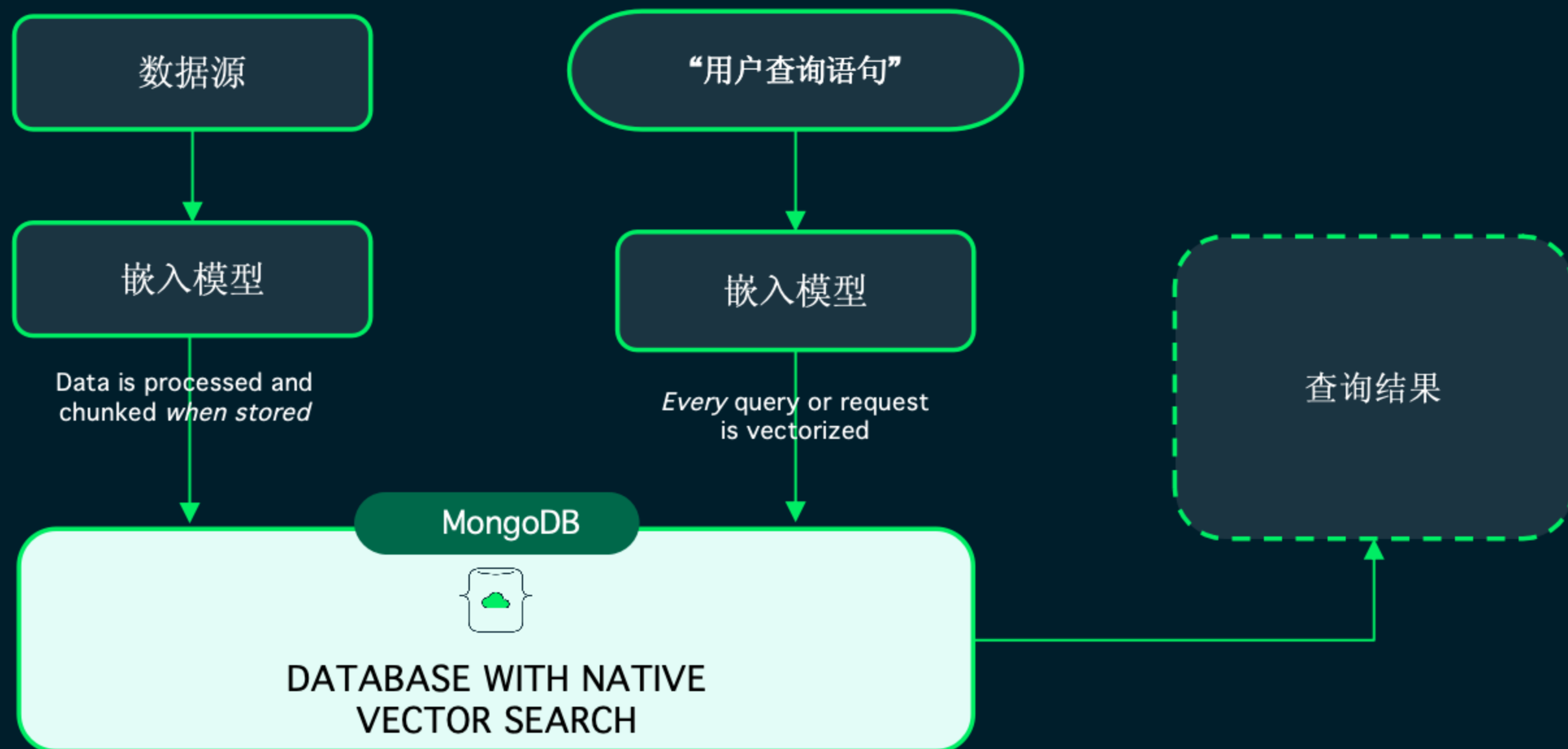


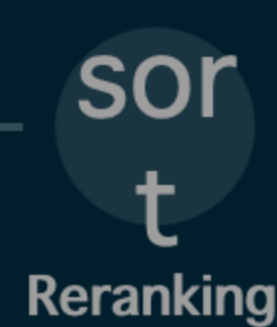
多模数据在一个平台中通过统一**API**来操作很方便，
但**精准高效的丰富查询**也很关键！





语义搜索过程





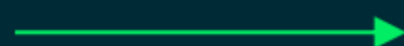
嵌入模型是将非结构化数据（如文本、图像、音频或视频）转换为称为向量（或嵌入）的长数字数组。这种数值表示允许应用程序捕捉数据的语义含义。

在MongoDB中，你可以使用任何Embedding模型，然后将Embedding后的数据手动保存到MongoDB中



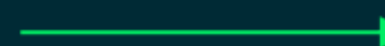
Data

(text, image, audio, video)



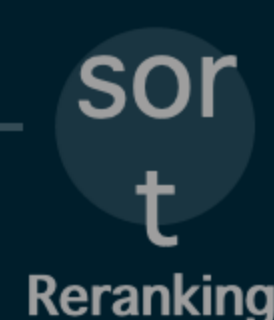
Embedding Model

(text, image, audio, video,
multimodal)



[0.3, 0.1, 0.2,, 0.4]

Vector



MongoDB 提供 Voyage AI

多种业界领先的 **Embedding** 和 **Reranking** 模型

Embedding Models

General-Purpose

Text
Lite | Standard | Large

Multimodal

Contextualized Chunk

Domain-Specific

Code

Legal

Finance

Rerankers

Text
Lite | Standard

VOYAGE AI

By  MongoDB.



Voyage AI是斯坦福大学**华人教授马腾宇团队**创建
业界领先的AI嵌入和重新排序模型公司
2025年被MongoDB收购



MongoDB 原生提供 Voyage AI

不仅模型领先，而且可以支持特色的 **非对称Embedding**

大幅度降低成本，提升性能，获得最佳性价比

文档离线场景

voyage-4-large 模型

特性：模型规模更大、向量更精准

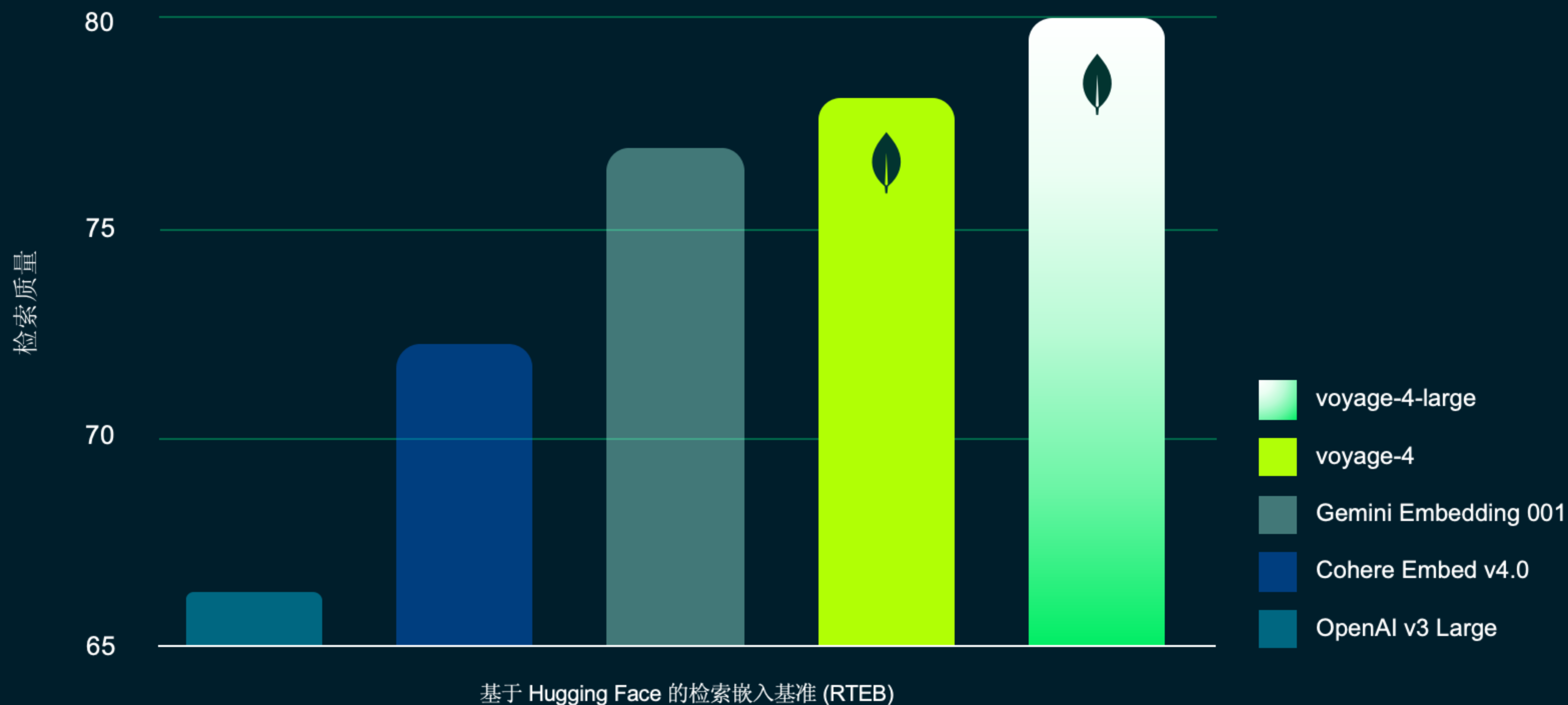
查询在线场景

voyage-4-lite 模型

特性：成本更便宜、响应速度更快



Voyage 的准确度处于行业领先水平



How to get embeddings with Anthropic

Anthropic does not offer its own embedding model. One embeddings provider that has a wide variety of options and capabilities encompassing all of the above considerations is Voyage AI.

Voyage AI makes state-of-the-art embedding models and offers customized models for specific industry domains such as finance and healthcare, or bespoke fine-tuned models for individual customers.

The rest of this guide is for Voyage AI, but you should assess a variety of embeddings vendors to find the best fit for your specific use case.

Available models

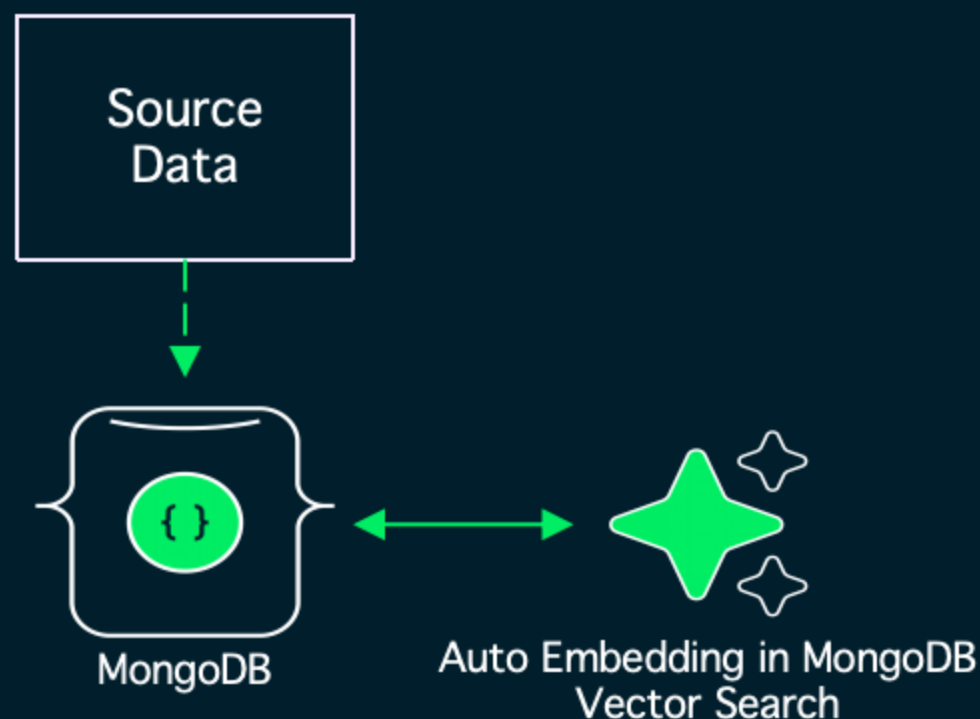
Voyage recommends using the following text embedding models:

Voyage 4 (latest generation)

Model	Context Length	Embedding Dimension	Description
voyage-4-large	32,000	1024 (default), 256, 512, 2048	The best general-purpose and multilingual retrieval quality. See blog post for details.
voyage-4	32,000	1024 (default), 256, 512, 2048	Optimized for general-purpose and multilingual retrieval quality. Balances quality and efficiency. See blog post for details.
voyage-4-lite	32,000	1024 (default), 256, 512, 2048	Optimized for latency and cost. See blog post for details.
voyage-4-nano	32,000	1024 (default), 256, 512, 2048	Open-weight model (Apache 2.0 license) available on Hugging Face. See blog post for details.

Auto-Embedding

降低AI工程复杂度



```
{
  type: "vectorSearch",
  definition: {
    "fields": [{
      "type": "autoEmbed",
      "modality": "text",
      "model": "voyage-4",
      "path": "content",
    }]
  }
}
```



直观的配置

开始使用向量搜索和嵌入模型现在就像点击按钮一样简单，极大降低的AI工程的复杂度意味着更快的价值实现速度



极简的架构

业务开发无需再管理独立的嵌入生成 workflow，只要以高吞吐量和低延迟的方式将**原始文本数据**写入 MongoDB，剩下的所有和向量相关的工作交给 MongoDB 完成



全自动 & 始终保持最新

在初始数据同步、文档操作和查询过程中自动生成 Embedding。随着数据的变化，Embedding 内容始终保持完美同步



实现了高效的 **AI** 驱动搜索

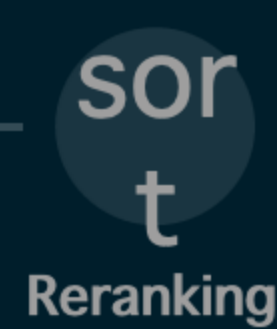
Auto Embedding in Vector Search

```
{
  "type": "vectorSearch",
  "definition": {
    "fields": [{
      "type": "autoEmbed",
      "modality": "text",
      "model": "voyage-4",
      "path": "content"
    }]
  }
}
```

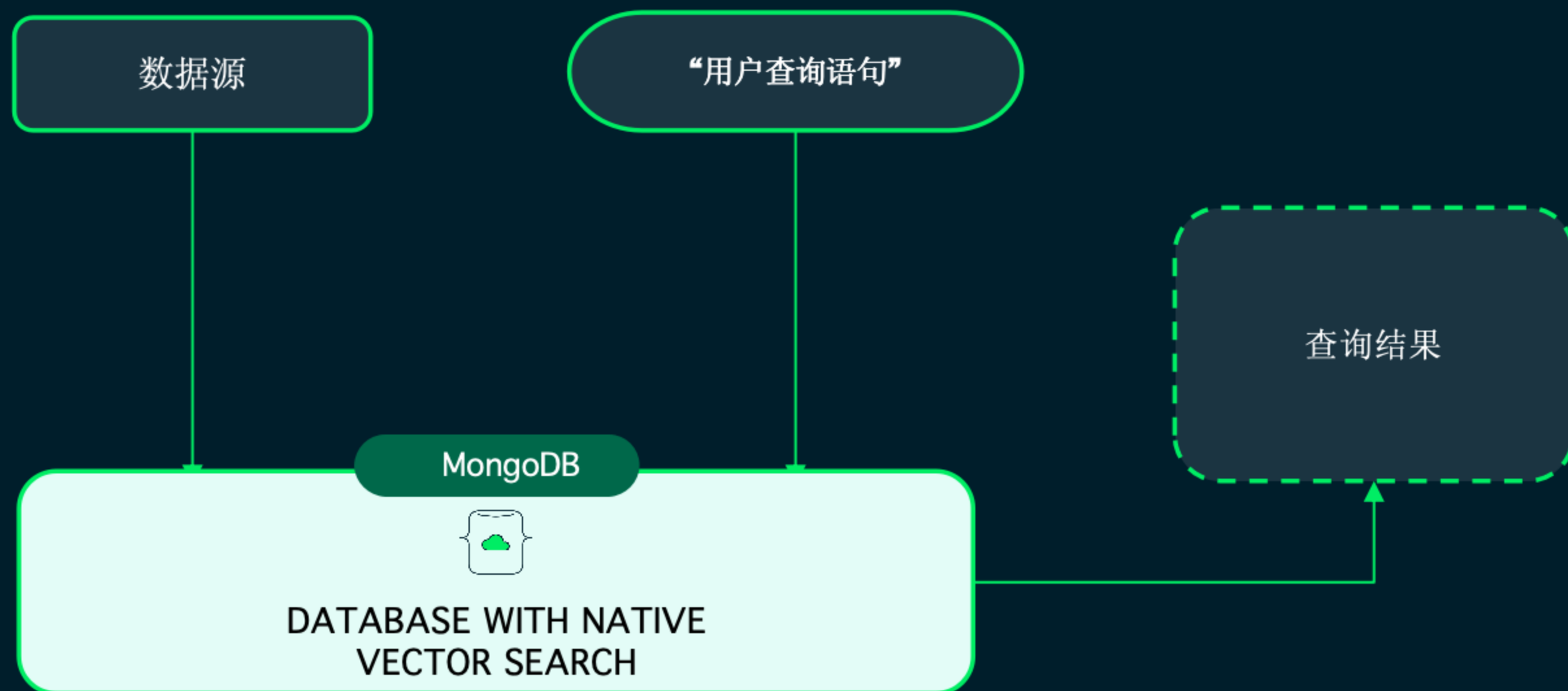
100
+

Auto Embedding Query

```
{
  "$vectorSearch": {
    "query": {"text": "<user-query>"},
    "model": "voyage-4-lite",
    "index": "indexName",
    "path": "content",
    "limit": 5,
    "numCandidates": 50,
  },
}
```



基于自动嵌入的语义搜索





轻松模型生命周期管理

MongoDB 的Auto-Embedding方案将模型升级从一项令人头疼的迁移工程转变为一次无缝的操作。

`add_circle_outline`

创建一个新的索引

升级到更新、性能更优的型号，并使用新的索引。

`settings_backup_restore`

后台全自动处理

索引创建过程无缝进行，不会干扰您的应用程序。

`sync`

同步更新

随着文档的更新，嵌入内容会自动保持同步。

`code_off`

不要写一行代码

没有额外的管道、脚本或迁移工作。

⚙️ Before (Manual Pipelines)



- Re-embed millions of docs manually
- Weeks of dev + infra overhead

After (With Atlas Auto-Embedding)



🔄 Auto-Reindex

- Choose new model – auto re-re-index
- Embeddings regenerated in background



Embedding

search
Search

sort
t

Reranking



MongoDB 向量特点

向量 Vector 和 业务数据
可以统一保存和处理

将高维向量嵌入与结构化业务元数据无缝融合在同一个 JSON 文档中，实现极简的架构与高性能查询。

```
// MongoDB Document Example
{
  "_id": "p_9527_human_warrior",
  // 1. 业务数据画像 (Business Data)
  "preferences": { "play_style": "hardcore_explorer", ... },
  "commercial_profile": { "buying_power_tier": "whale" },

  // 2. 嵌入向量 (Embeddings) - 存储在文档中
  "ai_embeddings": {
    "Preference_vector" : [0.12, -0.45, 0.78, ...],
    "buying_power_vector": [0.95, 0.02, ...]
  },

  // 3. 实时洞察
  "ai_insights": { "next_best_offer": "ancient_dragon_skin_pack" },
  "updated_at": "2026-04-19T08:15:00Z"
}
```

me
mo
ry
Embedding

sea
rch
Search

sor
t
Reranking



真实的多租户向量搜索业务往往都需要：**复杂的过滤和混合检索**



Pre-Filtering Search

过滤搜索

在向量查询前（或过程中），先对数据集进行过滤，确保结果集满足约束。



Hybrid Search

混合搜索

同时执行关键词BM25匹配与向量相似度计算，并通过两份结果按业务需求进行加权评分，确保结果兼顾精确与语义。

me
mo
ry
Embedding

sea
rch
Search

sor
t
Reranking



MognoDB 提供 非常丰富的pre-filter条件，实现更好的性能、更高的精度和更低的查询成本

MQL Operator	✅ Supported
<code>\$eq</code> , <code>\$ne</code>	✅
<code>\$gt</code> , <code>\$gte</code> , <code>\$lt</code> , <code>\$lte</code>	✅
<code>\$in</code> , <code>\$nin</code>	✅
<code>\$and</code> , <code>\$or</code> , <code>\$not</code>	✅
<code>\$exists</code> , <code>\$type</code>	✅

```
"$vectorSearch": {
  "filter": {
    "category": "sublets",           // equality
    "price": { "$lte": 2000 },       // comparison
    "available": true,               // boolean
    "borough": { "$in": ["Brooklyn", "Queens"] } // set membership
  }
}
```

Capability	<code>\$search.vectorSearch</code>
Fuzzy matching	✅
Phrase search	✅
Wildcards	✅
geoWithin	✅
Query strings	✅
Text analysis	✅

me
mo
ry
Embedding

sea
rch
Search

sor
t
Reranking



MongoDB 提供完善的 文本 与 向量 Hybrid Search 混合搜索功能



Vector Search

- ▮ 擅长处理开放式或模糊的问题
- ▮ 在高维数据（文档、图像、音频、视频）上表现极为高效

Hybrid Search

更高的准确性
高效处理复杂数据
更强大的原生搜索体验



Full-Text Search

- ▮ 快速高效地查找精确匹配结果
- ▮ 所需计算资源和成本更低

me
mo
ry
Embedding

sea
rch
Search

sor
t
Reranking



真实的业务向量搜索业务往往还需要： 高效率的检索



Stored Source

预存字段减少回表

在索引中预存源文档指定字段到 **mongodb**。查询时直接返回，减少数据库回表操作，以及WT缓存争用，显著提升QPS性能。



Flat index

穷举扫描适合强过滤

不走 HNSW 图，对过滤后的候选集做穷举扫描。特别适合多租户、每个租户向量数较少的场景。



Nested embeddings

子文档向量检索

将向量嵌在子文档数组中。允许直接对嵌套内容检索，并按最佳或平均匹配结果返回父文档。

me
mo
ry
Embedding

sea
rch
Search

sort
Reranking



Ranking Models (Rerankers)

Rerankers 给出一个查询和一组文档，根据与查询的相关性返回每个文档的排名。

这个重新排序过程可以帮助提高语义搜索结果的相关性，对结果进行评分和重新排序，以确保最大程度的相关性。

0.7 ●
0.8 ●
0.9 ●

Initial Results

● 0.9
● 0.8
● 0.7

Reranked Results

me
mo
ry
Embedding

sea
rch
Search

sort
Reranking



两阶段检索

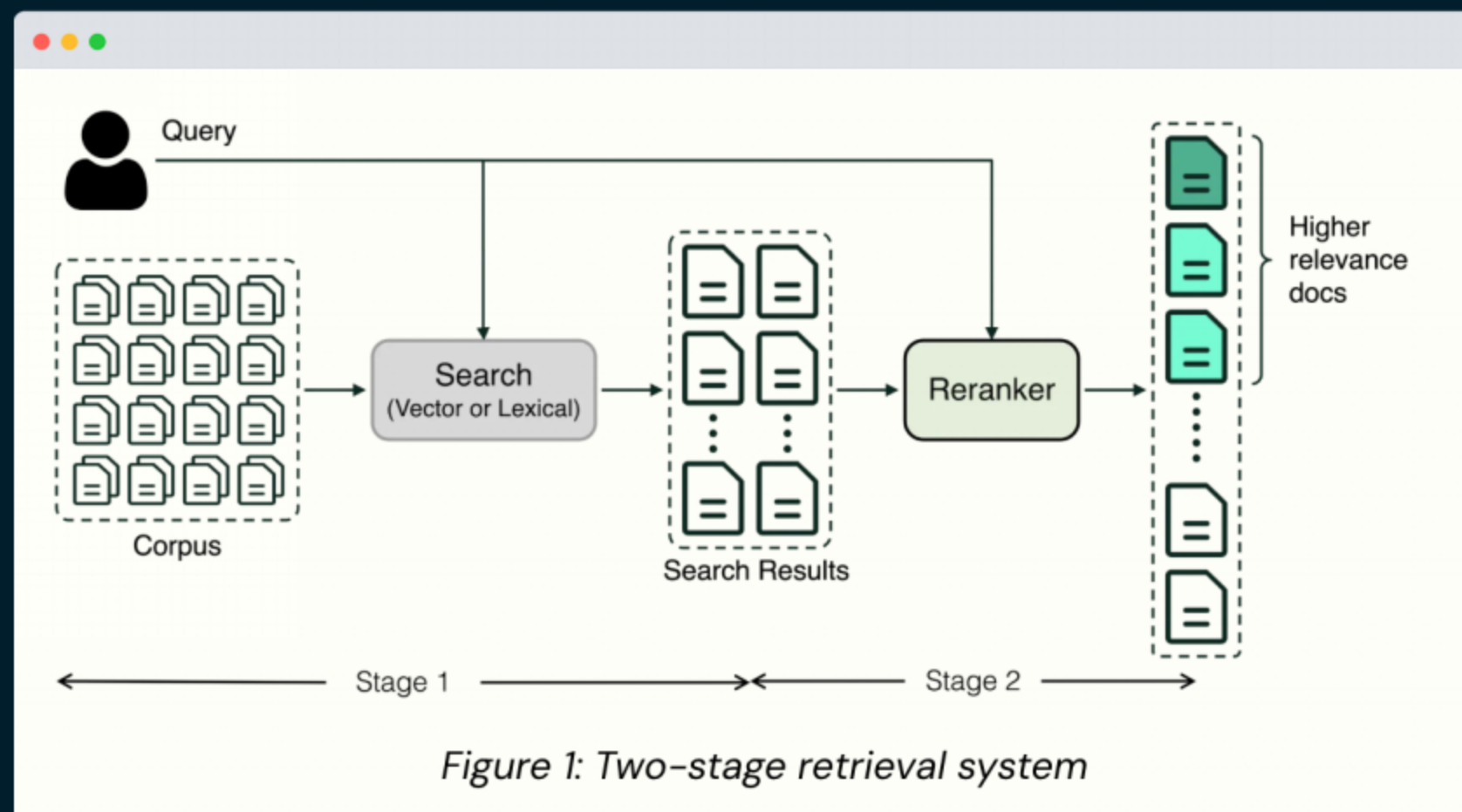
重排序器（Reranker）通常被用作两阶段检索系统中的精炼环节。

STAGE 01

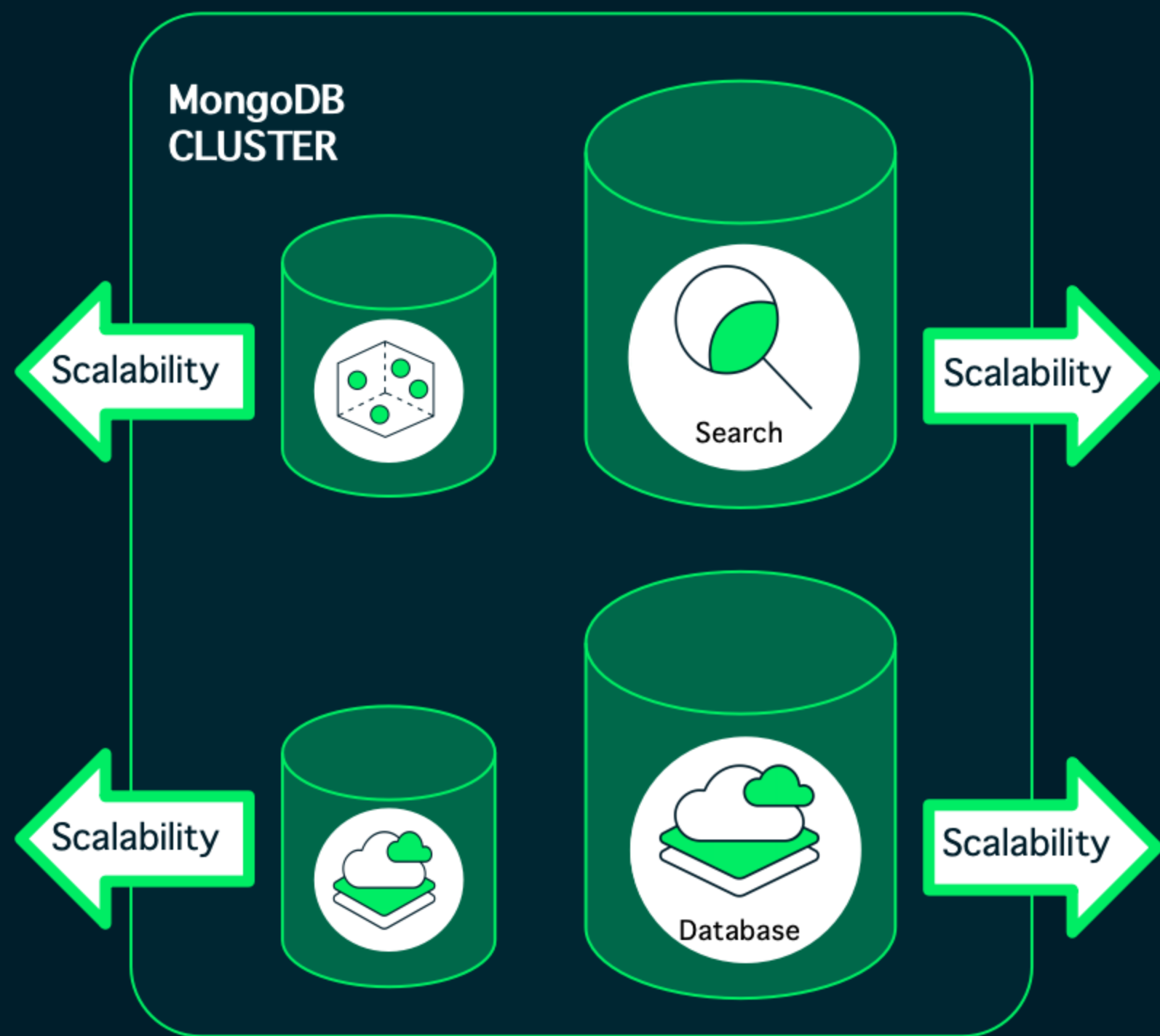
\$search / \$vectorSearch / \$rankFusion 会生成一系列广泛的初始搜索结果。

STAGE 02

重排序器通过评估查询与这些初始结果之间的相似度，从而生成重新排序后的结果。



专用 Search 节点



工作负载隔离

通过专属的搜索节点，从容应对并扩展高要求的搜索工作负载

规模化场景下更优的性能

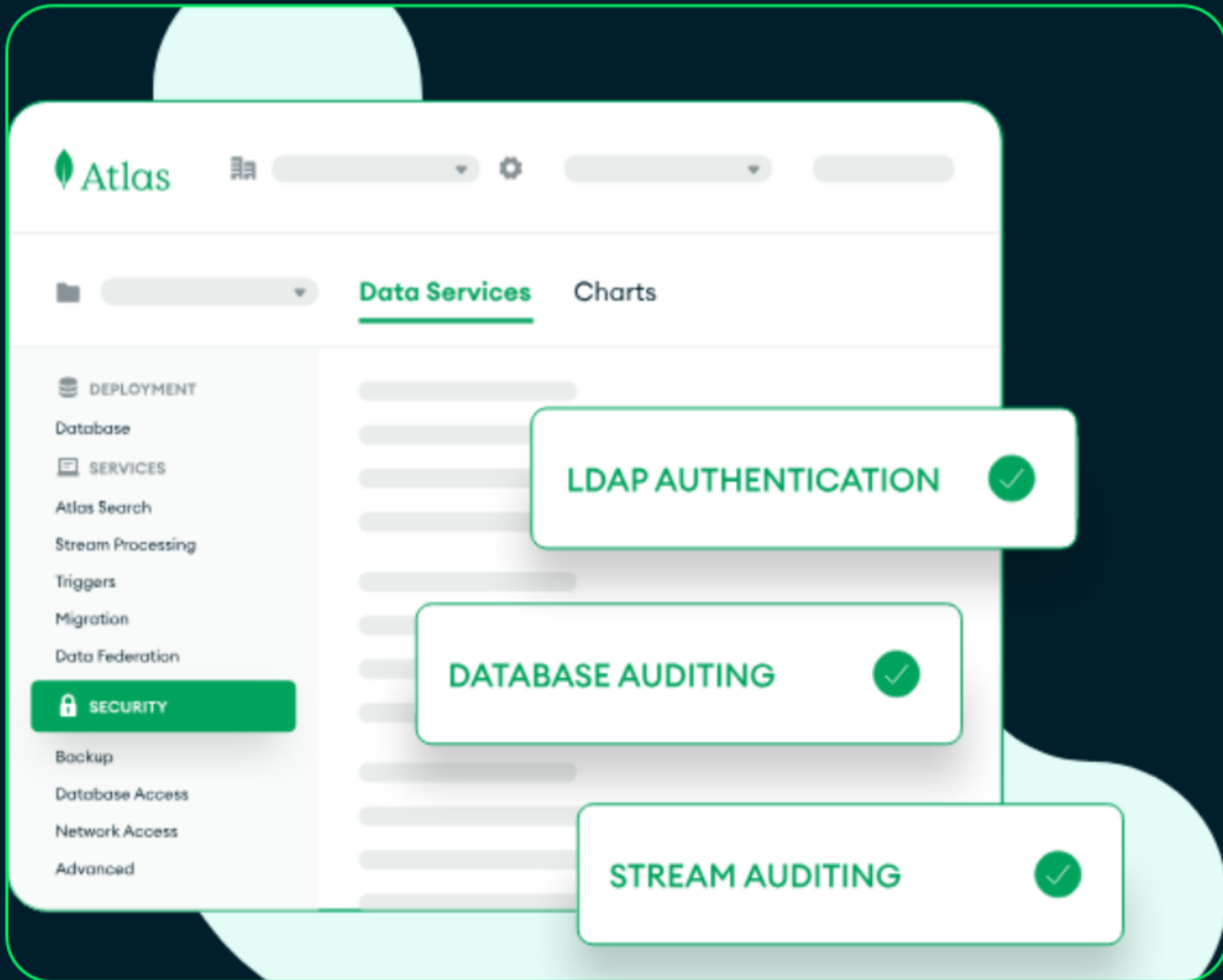
许多复杂查询的查询时间降低 40% - 60%

更高的可用性

工作负载隔离避免了数据库操作与搜索操作之间的资源争用



企业级就绪



业务运行在覆盖最广、全球分布的数据库上

- 无供应商锁定，在所有主流云提供商的 117+ 个区域中轻松扩缩容
- 所有能力都可以在本地部署

安心构建，信心十足

- 企业级安全性与数据隐私，满足严格的合规要求
- 深受技术最前沿、安全意识最强的组织的信赖



2024 年 10 月

7.0



8.0

36%

读吞吐量提升

56%

批量插入吞吐量提升



现已推出

2024 年 10 月

8.0



8.3

35%

写吞吐量提升

45%

读吞吐量提升

15%

事务吞吐量提升

在现代AI Agent生态系统中，框架和生态至关重要

从开发框架到云服务平台，MongoDB都有紧密的集成

APP DEV

Developer Frameworks & Code Assist

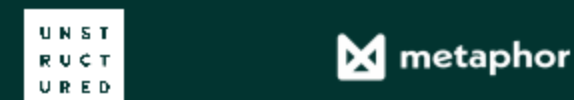


DATA

GenAI Safety



Data Integration

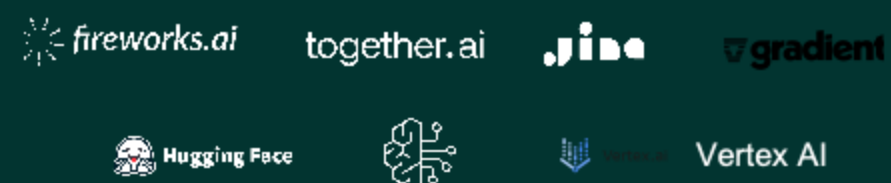


MODEL AND COMPUTE

Foundation & Embedding Models

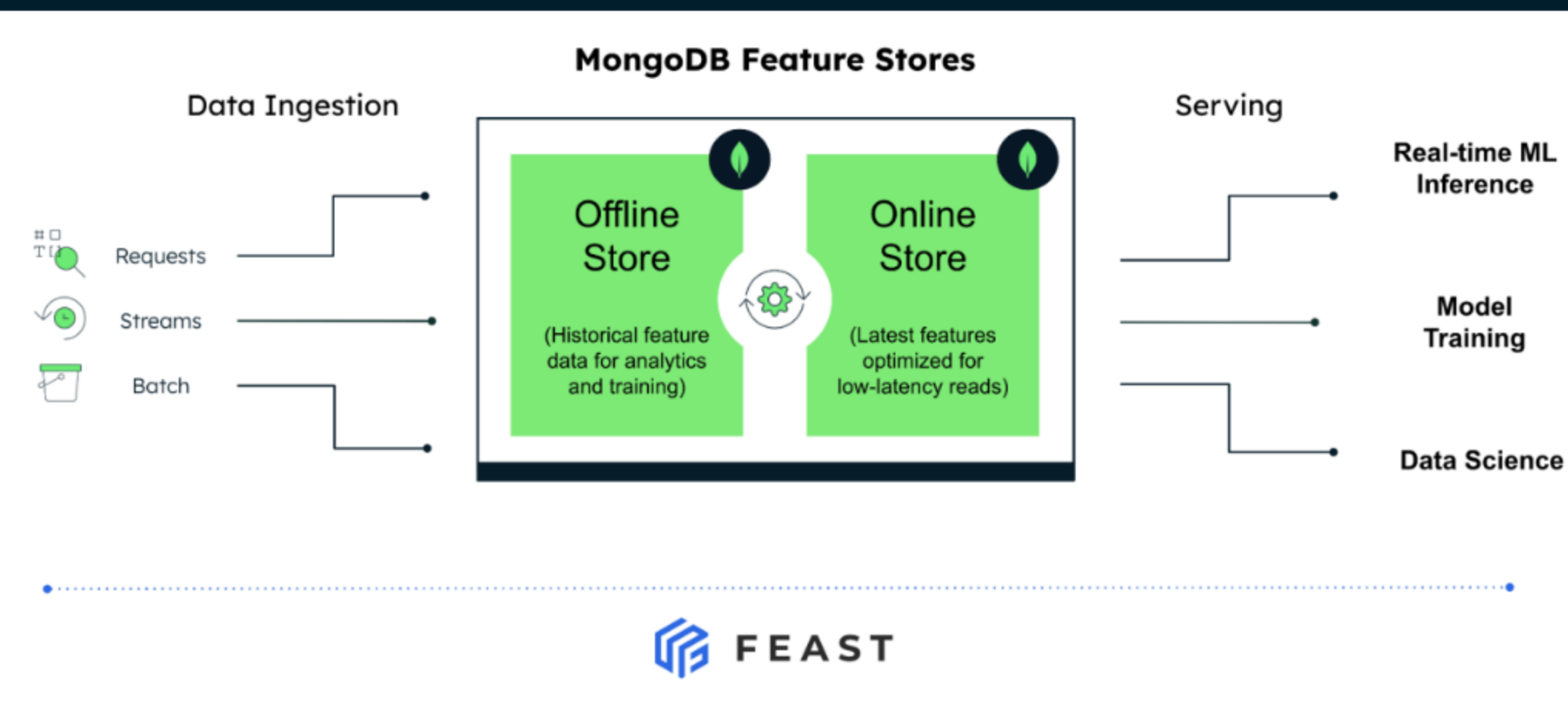


AI PaaS & Model Hosting



MongoDB + Feast: 统一的端到端AI训练特征数据平台

MongoBD 同时承担 Online + Offline Store 来做特征存储、检索和实时服务



```
project: my_feature_repo
registry: data/registry.db
provider: local

online_store:
  type: mongodb
  connection_string: "mongodb+srv://<user>:<password>@<cluster>.mongodb.net"
  database: "feast_online"

offline_store:
  type: mongodb
  connection_string: "mongodb+srv://<user>:<password>@<cluster>.mongodb.net"
  database: "feast_offline"

entity_key_serialization_version: 3
```

更多内容参见: <https://feast.dev/blog/mongodb-feast-integration/>

针对 AI Agent 开发，**框架和生态**也很重要



比如**LangGraph**的短期记忆Checkpoint和长期记忆Store，都可以直接调用MongoDB

```
from langgraph.checkpoint.mongodb import MongoDBSaver
from langgraph.graph import StateGraph
from pymongo import MongoClient

builder = StateGraph(int)
builder.add_node(add_one, lambda x: x + 1)
builder.set_entry_point(add_one)
builder.set_finish_point(add_one)
client = MongoClient(mongodb://localhost:27017)
memory = MongoDBSaver(client)
graph = builder.compile(checkpointer=memory)
config = {configurable: {thread_id: 1}}
```

针对 AI Agent 开发，框架和生态也很重要



比如专业的记忆管理框架 **Mem0**，也可以直接配置使用 **MongoDB**

```
from mem0 import Memory
os.environ["OPENAI_API_KEY"] = "sk-xx"

config = {
    "vector_store": {
        "provider": "mongodb",
        "config": { "db_name": "mem0-db", "collection_name": "mem0-collection", "mongo_uri": "mongodb://..." }
    }
}

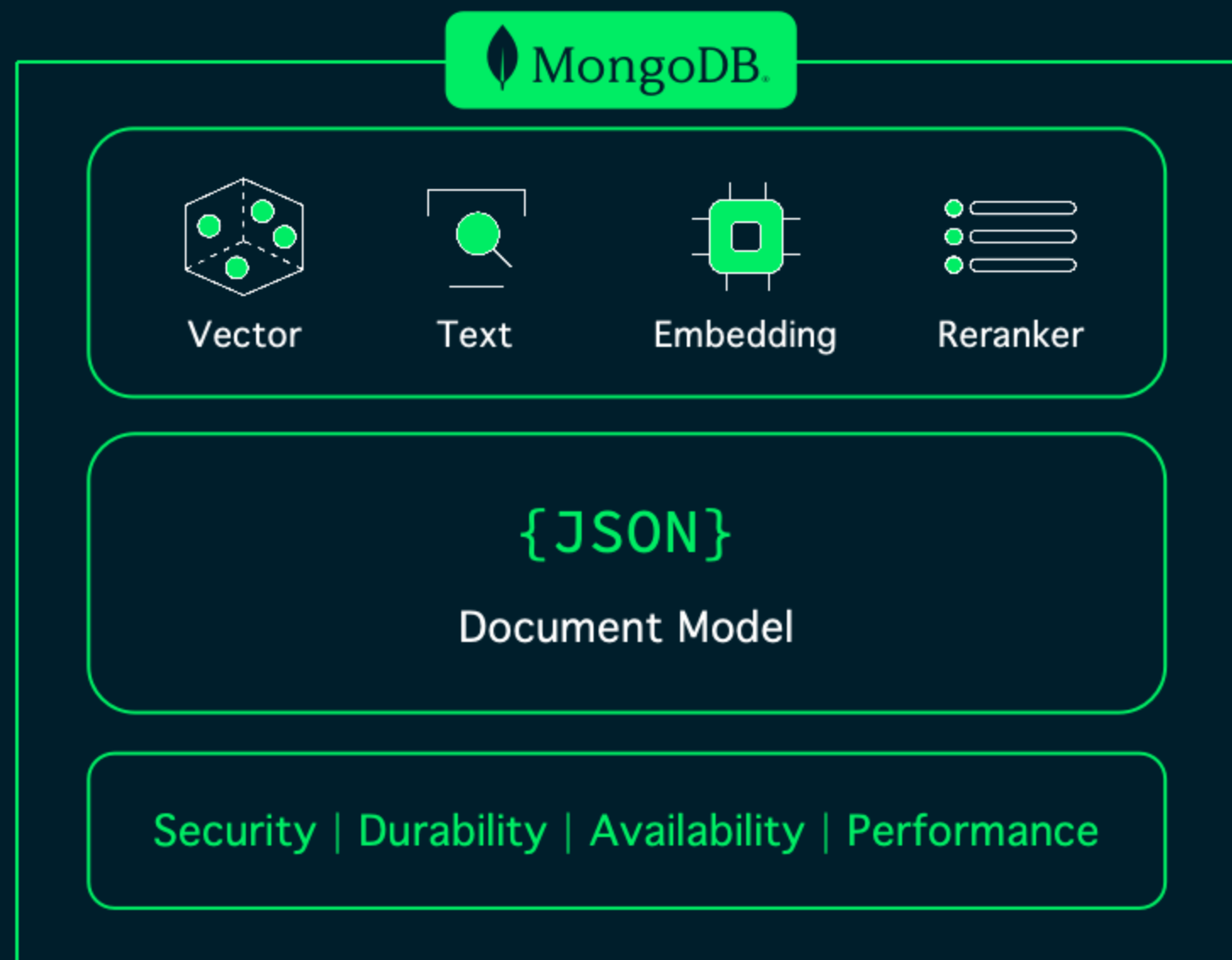
m = Memory.from_config(config)
messages = [
    {"role": "user", "content": "村长，铁匠铺在哪？"},
    {"role": "assistant", "content": "村东头，过桥就是，你是路痴，我带你吧。"},
    {"role": "user", "content": "谢谢！我这把剑能修吗？"},
    {"role": "assistant", "content": "老张手艺好，放心交给他。"}
]

m.add(messages, user_id="alice", metadata={"category": "npc_dialogue"})
```



MongoDB是AI领域的

数据基础





【AI实践 - 性能】

豆包对话 & 火山MongoDB：百万并发，毫秒可映射至缓存



【模型】豆包每一次文字对话，都有火山MongoDB的助力

【AI实践 - 存储检索】

豆包语音 & 火山MongoDB：让豆包春节语音亿次交互全程无断层



多模态一体化存储，解决音频交互的上下文衔接难题

0

多模态数据存储低时延

- 多模态数据与元信息存储在一起，相比传统存储方案降低10倍时延
- 多模态数据（文本、图片、音频等）以BinData存储，语音、图

【AI实践 - All in One】

抖音投稿 & 火山MongoDB：ALL in one典范，百TB创作数据统一存储零割裂





2026 MongoDB中国用户大会 上海站

时间: 2026年7月16日 (周四) 9:30-17:30

地点: 上海杨浦滨江万豪酒店

活动行



2026 MongoDB中国用户大会 上海站

浏览 6731 收藏 236

🕒 时间

07/16 周四 09:30 ~ 17:30

📍 地点

上海杨浦上海杨浦滨江万豪酒店

免费



THANKS

探索 AI 应用边界

Explore the limits of AI applications

AiCon

全球人工智能开发与应用大会

THANKS

探索 AI 应用边界

Explore the limits of AI applications

AiCon

全球人工智能开发与应用大会

THANKS

探索 AI 应用边界

Explore the limits of AI applications

AiCon

全球人工智能开发与应用大会

THANKS

探索 AI 应用边界

Explore the limits of AI applications

AiCon

全球人工智能开发与应用大会

THANKS

探索 AI 应用边界

Explore the limits of AI applications

AiCon

全球人工智能开发与应用大会

THANKS

探索 AI 应用边界

Explore the limits of AI applications

AiCon

全球人工智能开发与应用大会